

Berliner Hochschule für Technik

Bachelorarbeit

Konzeption und Implementierung einer interaktiven Visualisierung präemptiver Scheduling-Verfahren für die Lehre

Marten Kirschenhofer
Matrikelnummer: 833825
Studiengang: Medieninformatik

Betreuung: Prof. Dr. Christian Forler
Abgabedatum: 29.09.2026

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule für Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren.

Danksagung

Ich widme diese Arbeit meinen lieben Schwiegereltern, die mir stets mit gutem Rat zur Seite standen, meiner wundervollen Verlobten, die mich auch in schwierigen Zeiten unterstützt hat, und meinem besten Freund, bei dem ich trotz des großen Stresses stets ich selbst sein konnte.

Inhaltsverzeichnis

Vorwort	ii
Abbildungsverzeichnis	vi
Tabellenverzeichnis	vii
Verzeichnis der Listings	viii
1. Einleitung	1
1.1. Motivation und Problemstellung	1
1.2. Zielsetzung der Arbeit	1
1.3. Thematische Ausrichtung und Abgrenzung	2
1.4. Vorgehensweise und Aufbau der Arbeit	2
1.5. Leitgedanke der Arbeit	3
2. Grundlagen	4
2.1. Betriebssysteme und Prozessverwaltung	4
2.2. CPU-Scheduling in Betriebssystemen	9
2.3. Präemptive und nicht-präemptive Scheduling-Verfahren	10
2.4. Bewertungsmetriken	11
2.5. Scheduling-Verfahren	12
2.6. Vergleich der theoretischen Perspektiven	15
2.7. Algorithmusvisualisierung als Vermittlungsform	15
2.8. Accessibility als fachliche und gestalterische Grundlage	16
2.9. Zwischenfazit	16
3. Anforderungsanalyse	17
3.1. Zielgruppe und Einsatzkontext	17
3.2. Fachliche Anforderungen	17
3.3. Bereitstellung von Simulationsergebnissen	20
3.4. Didaktische Anforderungen	20
3.5. Interaktions- und Usability-Anforderungen	21
3.6. Vergleich mehrerer Simulationsläufe	22
3.7. Anforderungen an Accessibility	22
3.8. Technische Qualitätsziele	24
3.9. Abgeleitete funktionale und nicht-funktionale Anforderungen	24
4. Systementwurf	26
4.1. Entwurfsziele und Leitprinzipien	26
4.2. Gesamtarchitektur und Schichtenmodell	26
4.3. Komponenten und Verantwortlichkeiten	28
4.4. Datenfluss und Zustandsmodell	29
4.5. Interaktionsmodell für den Lernprozess	29

4.6.	HCI-Leitlinien im Entwurf	29
4.7.	Accessibility als Entwurfsprinzip	30
4.8.	Erweiterbarkeit und Entwurfsgrenzen	30
5.	Implementierung	31
5.1.	Technische Grundlagen und verwendete Technologien	31
5.2.	Datenmodell und Simulationskern	32
5.3.	Umsetzung der Scheduling-Verfahren	34
5.4.	Umsetzung der Visualisierung	36
5.5.	Interaktionsmechanismen und Nutzerworkflow	37
5.6.	Umsetzung der Accessibility-Anforderungen	37
5.7.	Build- und Deploymentprozess	38
5.8.	Besondere technische Herausforderungen	38
6.	Qualitätssicherung	39
6.1.	Qualitätsziele und Prüfstrategie	39
6.2.	Automatisierte Verifikation der Simulationslogik	39
6.3.	Prüfung von Interaktion und Accessibility	40
6.4.	Reproduzierbarkeit und Vergleichbarkeit	40
6.5.	Erreichte Testabdeckung und Grenzen	41
6.6.	Fazit zur Qualitätssicherung	41
7.	Ergebnis und Produktpräsentation	42
7.1.	Entwickeltes Endprodukt	42
7.2.	Produktüberblick und Nutzerfluss	42
7.3.	Produktpräsentation mit Screenshots	43
7.4.	Umgesetzte Funktionalität im Soll-Ist-Abgleich	43
7.5.	Accessibility-relevante Produktzustände	44
7.6.	Ergebnisbetrachtung	45
8.	Fazit und Ausblick	46
8.1.	Ausblick	46
	Literaturverzeichnis	47
A.	Anhang	48
A.1.	Ergänzende Abbildungen	48
A.2.	Zusätzliche Testfälle und Szenarien	48
A.3.	Operationalisierungsmatrix Accessibility	48
A.4.	Kompaktes manuelles Accessibility-Prüfprotokoll	49
A.5.	Ergänzende technische Artefakte (z. B. Konfigurationsbeispiele)	49

Abbildungsverzeichnis

4.1. Komponentenmodell der Anwendung mit Präsentationsschicht, Composables, Simulationskern und Domänenmodell.	27
5.1. Domänenmodell des Simulationslaufs mit seinen Ergebnisstrukturen.	33
5.2. Sequenzdiagramm eines Simulationslaufs von der Benutzeraktion bis zur Aktualisierung der Benutzeroberfläche.	35
5.3. Zustände und Übergänge eines Prozesses im Simulationsmodell.	36
7.1. Geladenes Szenario mit sichtbaren Task-Parametern als Grundlage für die Simulation.	43
7.2. Fokusansicht mit zeitlicher Ablaufdarstellung und Kennzahlenbereich für einen einzelnen Lauf.	43
7.3. Vergleichsansicht mehrerer Verfahren auf identischer Szenariobasis.	44

Tabellenverzeichnis

2.1. Für die Simulation verwendete Task-Zustände.	8
2.2. Ergänzende Perspektiven der verwendeten Betriebssystemliteratur.	15
3.1. Operationalisierte Accessibility-Anforderungen des fokussierten Prüfstands.	23

Verzeichnis der Listings

1. Einleitung

1.1. Motivation und Problemstellung

Scheduling-Verfahren gehören zu den zentralen Mechanismen moderner Betriebssysteme. Sie bestimmen, welche bereite Ausführungseinheit zu welchem Zeitpunkt Prozessorzeit erhält, und beeinflussen dadurch unter anderem die Reaktionsfähigkeit und Ressourcenauslastung eines Systems Stallings [2003], Tanenbaum and Bos [2016]. In vereinfachten Darstellungen wird dabei häufig von Prozessen gesprochen. In modernen Betriebssystemen erfolgt die konkrete Planung jedoch meist auf Ebene von Threads oder vergleichbaren Ausführungseinheiten.

Obwohl Scheduling-Verfahren in der Lehre formal beschrieben werden, bleibt ihr zeitlicher Ablauf für Studierende häufig abstrakt. Dies gilt insbesondere für präemptive Verfahren, bei denen laufende Ausführungseinheiten unterbrochen, erneut eingeplant und zwischen verschiedenen Zuständen bewegt werden. Der Unterschied zwischen den Verfahren lässt sich zwar theoretisch anhand ihrer Auswahlregeln erklären, wird jedoch in seiner konkreten Wirkung auf Ablauf, Wartezeit und Reaktionszeit erst durch eine zeitliche und interaktive Darstellung anschaulich.

Vor diesem Hintergrund wurde eine interaktive Webanwendung entwickelt, die präemptive Scheduling-Verfahren als steuerbaren Ablauf visualisiert. Die Anwendung soll die Auswahlentscheidungen des Schedulers, die Zustandsänderungen der Tasks und die daraus entstehenden zeitlichen Abläufe nachvollziehbar darstellen. Die Arbeit verfolgt dabei die Entwicklung eines didaktisch nutzbaren Softwaresystems, jedoch keine hypothesengetriebene Wirksamkeitsstudie mit Studierenden.

1.2. Zielsetzung der Arbeit

Ziel dieser Arbeit ist die Konzeption und Implementierung einer interaktiven Webanwendung zur Veranschaulichung präemptiver Scheduling-Verfahren. Die Anwendung soll:

- ausgewählte Scheduling-Verfahren korrekt und deterministisch simulieren,
- die zeitliche Dynamik von Ankunft, Ausführung, Präemption und Abschluss verständlich visualisieren,
- die Verfahren auf einer identischen Szenariobasis vergleichbar machen,

- relevante Bewertungsmetriken wie Wartezeit, Durchlaufzeit, Reaktionszeit und Durchsatz darstellen,
- die Entscheidungswege des Schedulers durch Zustands- und Ereignisinformationen nachvollziehbar machen,
- und als ergänzendes Lehr- und Lernmittel im Kontext der Betriebssystemlehre einsetzbar sein.

Die Arbeit verbindet hierfür Grundlagen der Betriebssysteme und des CPU-Schedulings mit Prinzipien des Software Engineering, der interaktiven Visualisierung und der Barrierefreiheit. Der Schwerpunkt liegt auf der fachlich korrekten Simulation, der nachvollziehbaren Darstellung und der technischen Qualität der entwickelten Anwendung.

1.3. Thematische Ausrichtung und Abgrenzung

Die Arbeit ist als engineering-orientierte Produktdokumentation angelegt. Gegenstand ist die Entwicklung und Begründung einer interaktiven Webanwendung einschließlich ihrer fachlichen, technischen und gestalterischen Entscheidungen.

Die Anwendung stellt Scheduling-Verfahren in einem vereinfachten Simulationsmodell dar. Ein simuliertes Objekt wird dabei als *Task* bezeichnet und besitzt unter anderem eine Ankunftszeit, eine benötigte Ausführungszeit, eine Priorität und einen aktuellen Zustand. Die Simulation bildet nicht alle Eigenschaften realer Betriebssysteme ab. Insbesondere werden keine realen Betriebssystemprozesse erzeugt, keine tatsächlichen Hardware-Ressourcen verwaltet und keine realen Kontextwechsel ausgeführt. Stattdessen werden die für das Verständnis von CPU-Scheduling relevanten Zustände, Ereignisse und Zeitabläufe modelliert.

Die Arbeit ist von empirischen Wirksamkeitsstudien mit Nutzergruppen, statistischen Auswertungen und Hypothesentests abgegrenzt. Es wird daher nicht untersucht, ob die Anwendung im Vergleich zu anderen Lehrmethoden nachweisbar zu besseren Lernergebnissen führt. Im Fokus stehen stattdessen die Simulationskonsistenz, die Architekturentscheidungen, die Implementierung, die Interaktion und die qualitätsgesicherte Produktdarstellung.

1.4. Vorgehensweise und Aufbau der Arbeit

Die Vorgehensweise umfasst sechs aufeinander aufbauende Schritte:

1. Erarbeitung der fachlichen Grundlagen zu Betriebssystemen, CPU-Scheduling und Bewertungsmetriken,
2. Ableitung funktionaler und nicht-funktionaler Anforderungen,
3. Entwicklung des Systementwurfs mit Architektur- und Interaktionsmodell,
4. Implementierung der Webanwendung und der Scheduling-Verfahren,

5. Qualitätssicherung der Simulationslogik, der Benutzerinteraktion und der Barrierefreiheit,
6. Darstellung und Einordnung der entwickelten Anwendung.

Kapitel 2 behandelt die theoretischen Grundlagen zu Betriebssystemen, Prozessen, Threads und CPU-Scheduling. Kapitel 3 beschreibt die Anforderungsanalyse. Der Systementwurf mit den architektonischen und interaktionsbezogenen Entscheidungen wird in Kapitel 4 dargestellt. Kapitel 5 behandelt die Implementierung der Webanwendung. Kapitel 6 beschreibt die Qualitätssicherung und die durchgeführten Prüfungen. Kapitel 7 präsentiert die entwickelte Anwendung und ordnet ihre Ergebnisse ein. Kapitel 8 schließt die Arbeit mit einem Fazit und einem Ausblick ab.

1.5. Leitgedanke der Arbeit

Der leitende Gedanke dieser Arbeit lautet:

Praktische Verständlichkeit entsteht durch das Zusammenspiel aus fachlicher Korrektheit, klarer Architektur, transparenter Interaktion und nachvollziehbarer Ablaufdarstellung.

Die Scheduling-Visualisierung wird damit nicht ausschließlich als Implementierungsaufgabe verstanden. Sie verbindet Fachwissen über Betriebssysteme mit Softwaretechnik, interaktiver Visualisierung und didaktischer Gestaltung. Die entwickelte Anwendung soll Studierende dabei unterstützen, die Abläufe präemptiver Scheduling-Verfahren nachzuvollziehen und deren theoretische Konzepte anhand konkreter Szenarien greifbar zu machen. Eine empirische Aussage über den tatsächlichen Lernerfolg ist damit nicht verbunden und bleibt einer möglichen weiterführenden Untersuchung vorbehalten.

2. Grundlagen

Dieses Kapitel stellt die theoretischen Grundlagen dar, auf denen die Konzeption, Implementierung und Bewertung der Anwendung basieren. Zunächst werden die Aufgaben moderner Betriebssysteme sowie die Begriffe Prozess und Thread erläutert. Anschließend werden die für den Visualizer relevanten Task-Zustände, Scheduling-Konzepte und Bewertungsmetriken beschrieben. Abschließend wird der Zusammenhang zwischen diesen Grundlagen und der Visualisierung von Scheduling-Verfahren hergestellt.

2.1. Betriebssysteme und Prozessverwaltung

Um die Notwendigkeit und den Nutzen der dieser Arbeit zugrunde liegenden Konzepte des entwickelten Visualizers zu erläutern, ist zunächst die Rolle moderner Betriebssysteme und ihrer Prozessverwaltung zu betrachten. Moderne Betriebssysteme vermitteln zwischen der Hardware und den darauf ausgeführten Anwendungsprogrammen. Sie verbergen die technische Komplexität der Hardware weitgehend vor den Nutzenden und stellen standardisierte Schnittstellen für die Programmausführung und Ressourcenverwaltung bereit. Dadurch können Anwendungen verwendet werden, ohne dass die technischen Details der zugrunde liegenden Hardware bekannt sein müssen.

Nach Tanenbaum und Bos lassen sich Betriebssysteme aus zwei ergänzenden Perspektiven betrachten: als erweiterte Maschine beziehungsweise Abstraktionsebene und als Ressourcenverwalter Tanenbaum and Bos [2016]. Diese beiden Perspektiven bilden den konzeptionellen Rahmen für die folgenden Betrachtungen.

Als Abstraktionsebene verbergen Betriebssysteme die technischen Einzelheiten der Hardware und stellen einheitliche Konzepte wie Dateien, virtuelle Speicheradressen, Prozesse und standardisierte Geräteschnittstellen bereit. Anwendungen müssen dadurch beispielsweise nicht die physische Organisation eines Speichermediums kennen, um eine Datei zu öffnen oder zu speichern. Diese Abstraktion wird durch Systemaufrufe und Bibliotheken ermöglicht, die den direkten Hardwarezugriff kapseln. Die Abstraktionsebene erleichtert die Entwicklung von Anwendungssoftware, da sie eine einheitliche Schnittstelle bereitstellt und die Komplexität der Hardware verbirgt.

Gleichzeitig koordinieren Betriebssysteme den Zugriff auf begrenzte Hardware-Ressourcen. Zu diesen Ressourcen zählen insbesondere Prozessorzeit, Arbeitsspeicher, Speicherplatz, Dateien, Ein-/Ausgabegeräte und Netzwerkverbindungen. Das Betriebssystem weist die-

se Ressourcen den laufenden Prozessen und Threads zu, kontrolliert Zugriffsrechte und behandelt Ereignisse wie Unterbrechungen oder Ein-/Ausgabeoperationen. Für die Einteilung der Prozessorzeit ist das sogenannte CPU-Scheduling vorgesehen. Dabei wird entschieden, zu welchem Zeitpunkt welche Ausführungseinheit wie lange Prozessorzeit erhält.

2.1.1. Prozesse

Ein Prozess kann als ein Programm in Ausführung bezeichnet werden. Während ein Programm zunächst nur eine statische Sammlung von Anweisungen darstellt, erhält es durch seine Ausführung einen vom Betriebssystem verwalteten Zustand. Dieser umfasst unter anderem den Programmcode, den virtuellen Adressraum, Registerinhalte, den Program Counter, geöffnete Dateien sowie weitere Ressourcen. Außerdem besitzt jeder Prozess eine eindeutige Prozesskennung und Verwaltungsinformationen zu seinem aktuellen Lebenszyklus.

Ein Prozess stellt dabei einen geschützten Rahmen für die Ausführung eines Programms dar. Das Betriebssystem kann Prozesse dadurch unabhängig voneinander verwalten, anhalten, fortsetzen und beenden. In modernen Betriebssystemen wird die konkrete Ausführung häufig auf Ebene der Threads betrachtet, da Threads die eigentlichen Ausführungseinheiten darstellen. Der Prozess bleibt jedoch die zentrale Einheit für die Verwaltung des Adressraums und vieler gemeinsam genutzter Ressourcen Stallings [2003], Tanenbaum and Bos [2016].

Prozesse wurden als Abstraktion eingeführt, um mehrere Programme kontrolliert und möglichst unabhängig voneinander verwalten zu können. In frühen Stapelverarbeitungssystemen wurden Aufträge überwiegend nacheinander abgearbeitet. Mit der Entwicklung von Mehrprogrammsystemen konnte ein Prozess die CPU abgeben, während er beispielsweise auf eine Ein-/Ausgabe wartete. Das Betriebssystem konnte in dieser Zeit eine andere bereitete Ausführungseinheit ausführen. Dadurch wurde die Auslastung der Rechenressourcen verbessert und die Grundlage für die später verbreiteten Time-Sharing-Systeme geschaffen. Ein frühes Beispiel dafür ist das am Massachusetts Institute of Technology entwickelte Compatible Time-Sharing System (CTSS), das 1961 demonstriert wurde Computer History Museum [1961].

Die Prozessabstraktion bietet mehrere Vorteile. Erstens ermöglicht sie die kontrollierte Ausführung mehrerer Programme und damit eine bessere Auslastung der CPU. Zweitens unterstützt sie die Isolation von Speicherbereichen und Ressourcen. Fehler eines Prozesses müssen dadurch nicht unmittelbar andere Prozesse beeinträchtigen. Drittens kann das Betriebssystem die Prozessorzeit anhand von Scheduling-Regeln auf mehrere Prozesse beziehungsweise Threads verteilen. Auf Mehrkernprozessoren können mehrere Ausführungseinheiten außerdem gleichzeitig auf unterschiedlichen Prozessorkernen ausgeführt werden.

Auf einem Einkernprozessor kann zu einem Zeitpunkt jedoch nur ein Ausführungsstrang tatsächlich CPU-Zeit beanspruchen. Mehrere Prozesse oder Threads können dort nicht

physisch gleichzeitig ausgeführt werden. Durch schnelle Wechsel zwischen den bereiten Ausführungseinheiten entsteht für die Nutzenden dennoch der Eindruck einer gleichzeitigen Ausführung.

Prozesse sind vergleichsweise schwergewichtige Verwaltungseinheiten. Ihre Erzeugung und Beendigung sowie der Wechsel zwischen Prozessen verursachen Verwaltungsaufwand. Außerdem sind Prozesse durch ihre getrennten virtuellen Adressräume stärker voneinander isoliert. Der Datenaustausch zwischen ihnen erfordert daher spezielle Kommunikationsmechanismen wie Pipes, Nachrichtenwarteschlangen oder gemeinsam genutzten Speicher. Für Programme, die mehrere eng zusammenhängende Aufgaben parallel bearbeiten sollen, kann die ausschließliche Verwendung mehrerer Prozesse daher unnötig aufwendig sein.

2.1.2. Threads als Ausführungseinheiten

Um mehrere eng zusammenhängende Aufgaben innerhalb eines gemeinsamen Prozesskontexts effizient ausführen zu können, wurden Threads eingeführt. Ein Thread ist ein einzelner Ausführungsstrang innerhalb eines Prozesses. Während der Prozess den geschützten Rahmen mit seinem virtuellen Adressraum und den zugehörigen Ressourcen beschreibt, repräsentiert der Thread die konkrete Ausführung von Programmanweisungen.

Ein Prozess kann aus einem oder mehreren Threads bestehen. Threads desselben Prozesses teilen sich typischerweise den Programmcode, den Heap, globale Variablen, geöffnete Dateien und weitere Ressourcen. Jeder Thread verfügt dagegen über einen eigenen Program Counter, eigene Register, einen eigenen Stack und einen eigenen Ausführungszustand Tanenbaum and Bos [2016].

Die gemeinsame Nutzung von Ressourcen erleichtert die Kommunikation zwischen Threads und verursacht in der Regel weniger Verwaltungsaufwand als die Kommunikation zwischen voneinander isolierten Prozessen. Threads können dadurch unterschiedliche Aufgaben innerhalb eines Programms übernehmen. In einem Browser kann beispielsweise ein Thread die Benutzeroberfläche bedienen, während andere Threads Netzwerkdaten verarbeiten oder Downloads durchführen.

Auf einem Einkernprozessor wird zu jedem Zeitpunkt nur ein Thread tatsächlich ausgeführt. Der Scheduler teilt die Prozessorzeit durch schnelle Wechsel zwischen den Threads auf. Auf einem Mehrkernprozessor können mehrere Threads desselben Prozesses dagegen gleichzeitig auf unterschiedlichen Prozessorkernen ausgeführt werden.

Die gemeinsame Nutzung des Speichers bringt neben Vorteilen auch Risiken mit sich. Greifen mehrere Threads ohne geeignete Synchronisation auf dieselben Daten zu, kann eine Race Condition entstehen. Dabei hängt das Ergebnis von der zeitlichen Reihenfolge der Zugriffe ab. Synchronisationsmechanismen wie Mutexes, Semaphore oder Monitore können solche Zugriffe koordinieren. Eine fehlerhafte Synchronisation kann jedoch beispielsweise zu einem Deadlock führen, bei dem Threads dauerhaft aufeinander warten.

Threads verbessern daher die Reaktionsfähigkeit und Parallelität von Programmen, erfordern aber eine sorgfältige Verwaltung gemeinsam genutzter Daten. Die Prozessstruktur sorgt dabei für einen übergeordneten Schutz- und Ressourcenrahmen, während Threads die eigentlichen Ausführungsstränge innerhalb dieses Rahmens darstellen.

2.1.3. Aufgaben des Betriebssystems

Die Funktionen eines Betriebssystems lassen sich unter anderem in Prozessverwaltung, Speicherverwaltung, Ein-/Ausgabeverwaltung, Dateiverwaltung, Geräteverwaltung, Netzwerkverwaltung und Sicherheitsfunktionen gliedern.

Die Prozessverwaltung umfasst die Erzeugung, Zustandsverwaltung, Ausführung und Beendigung von Prozessen und Threads sowie deren Kommunikation. Die Speicherverwaltung stellt Prozessen jeweils einen eigenen virtuellen Adressraum bereit. Threads desselben Prozesses teilen sich dagegen grundsätzlich diesen Adressraum, verfügen jedoch über eigene Stacks und Register. Die Speicherverwaltung sorgt außerdem dafür, dass belegter Speicher wieder freigegeben wird und Prozesse nicht unerlaubt auf den Speicher anderer Prozesse zugreifen.

Die Datei- und Geräteverwaltung abstrahieren den Zugriff auf dauerhafte Daten und angeschlossene Hardware. Anwendungen können dadurch mit Dateien und Geräten arbeiten, ohne deren technische Implementierung im Detail kennen zu müssen. Ergänzend schützen Benutzer- und Berechtigungsmodelle den Zugriff auf Ressourcen.

Für das CPU-Scheduling sind vor allem die Prozess- und Threadverwaltung sowie die Unterbrechungsbehandlung relevant. Das Betriebssystem verwaltet Ausführungskontexte, führt Zustandswechsel durch und reagiert auf Ereignisse wie Timer-Interrupts, Ein-/Ausgabeabschlüsse oder die Ankunft neuer Prozesse und Threads.

Auf einem Einkernprozessor kann zu einem Zeitpunkt nur ein Thread tatsächlich ausgeführt werden. Durch schnelle Wechsel zwischen bereiten Threads entsteht dennoch der Eindruck einer gleichzeitigen Ausführung. Auf Mehrkernprozessoren können mehrere Threads oder Prozesse tatsächlich parallel auf unterschiedlichen Kernen laufen. Auch dann muss das Betriebssystem die Ausführungseinheiten auf die verfügbaren Kerne verteilen und die Nutzung der Ressourcen überwachen.

Stallings beschreibt Betriebssysteme insbesondere aus der Perspektive der Verwaltung und Zuteilung von Ressourcen. Tanenbaum und Bos ordnen dieselben Aufgaben stärker in ein konzeptionelles Modell aus Prozessen, Threads, Zuständen und Betriebssystemkomponenten ein. Für diese Arbeit werden beide Perspektiven verbunden: Die Ressourcenperspektive erklärt, warum Scheduling erforderlich ist, während das Prozess- und Threadmodell beschreibt, wie die daraus entstehenden Entscheidungen umgesetzt werden Stallings [2003], Tanenbaum and Bos [2016].

Für die theoretische Beschreibung realer Betriebssysteme werden im Folgenden weiter-

hin die Begriffe Prozess und Thread verwendet. Für das konkrete Simulationsmodell des Visualizers wird dagegen die einheitliche Bezeichnung *Task* eingeführt. Ein Task repräsentiert dort eine schedulierbare Ausführungseinheit mit eigener Ankunftszeit, Burst-Zeit, Priorität und aktuellem Zustand.

Im Betriebssystemkontext kann eine solche Ausführungseinheit je nach Modellierung einem Prozess oder einem Thread entsprechen. Der Visualizer abstrahiert diese technische Unterscheidung, da für die Simulation vor allem die zeitliche Zuteilung der CPU und die daraus entstehenden Zustandsänderungen relevant sind. Die Simulation modelliert somit keine innerhalb eines Prozesses parallel laufenden Threads, sondern unabhängige Tasks, deren Ausführung durch das jeweils gewählte Scheduling-Verfahren gesteuert wird.

2.1.4. Task-Zustände und Zustandsübergänge

Ein Task befindet sich zu jedem Zeitpunkt in einem bestimmten Simulationszustand. Für die Modellierung des Visualizers werden die Zustände *pending*, *ready*, *running*, *preempted* und *finished* verwendet. Diese Zustandsbezeichnungen bilden die fachliche Modellierung der Anwendung ab und sind nicht als vollständige Abbildung aller Zustände realer Betriebssysteme zu verstehen.

Ein neu angelegter Task ist zunächst noch nicht bereit, solange seine Ankunftszeit nicht erreicht ist. Danach wird er in die Bereitschaftsmenge aufgenommen. Der Zustand *running* bezeichnet den Task, dem aktuell CPU-Zeit zugewiesen ist. Wird ein laufender Task durch einen anderen verdrängt, wird dies in der Simulation als Übergang in den Zustand *preempted* erfasst. Nach einer erneuten Auswahl kann er weiterlaufen. Mit dem Abschluss der benötigten Burst-Zeit erreicht der Task den Zustand *finished*.

Diese Übergänge bilden die fachliche Grundlage für die Ereignisse des Simulationslaufs und für die Segmentierung der Gantt-Ansicht.

Tabelle 2.1.: Für die Simulation verwendete Task-Zustände.

Zustand	Bedeutung	Typischer Übergang
<i>pending</i>	Task ist noch nicht angekommen.	Ankunftszeit erreicht → <i>ready</i>
<i>ready</i>	Task wartet auf CPU-Zuteilung.	Auswahl durch den Scheduler → <i>running</i>
<i>running</i>	Task verwendet die CPU.	Abschluss, Präemption oder Quantumablauf
<i>preempted</i>	Task wurde unterbrochen und wartet auf erneute Auswahl.	Erneute Auswahl → <i>running</i>
<i>finished</i>	Task hat seine benötigte Ausführungszeit verbraucht.	Endzustand

2.1.5. Scheduler, Dispatcher und Kontextwechsel

Der Scheduler entscheidet, welcher bereite Task als Nächstes CPU-Zeit erhalten soll. Der Dispatcher setzt diese Entscheidung technisch um, indem er den Kontext des bisher laufenden Tasks sichert und den Kontext des ausgewählten Tasks lädt. Zum Ausführungskontext gehören je nach System unter anderem Registerinhalte, der Program Counter, der Stack sowie weitere Verwaltungsinformationen.

Der Wechsel zwischen zwei Ausführungseinheiten wird als Kontextwechsel bezeichnet. Er ermöglicht unter anderem Präemption, kann aber auch durch die Beendigung oder Blockierung eines Tasks ausgelöst werden. Dabei wird der bisherige Zustand gesichert und ein anderer Zustand wiederhergestellt. Dieser Vorgang verursacht Verwaltungsaufwand und kann die tatsächlich für Anwendungsaufgaben verfügbare Prozessorzeit reduzieren.

In der Simulation wird der Aufwand eines Kontextwechsels nicht als zusätzliche reale Prozessorzeit modelliert. Die Anzahl der Kontextwechsel wird jedoch als Kennzahl erfasst. Dadurch kann sichtbar gemacht werden, dass ein Verfahren zwar eine kurze Reaktionszeit erreichen, zugleich aber mehr Kontextwechsel verursachen kann.

Ein Scheduling-Ereignis kann durch mehrere Ursachen ausgelöst werden. Dazu gehören die Ankunft eines neuen Tasks, der Ablauf eines Zeitquantums, die Beendigung eines Tasks oder ein Wechsel von einer blockierten in eine bereite Situation. Die Implementierung abstrahiert diese Betriebssystemereignisse auf diskrete Zeitschritte. Diese Abstraktion ist für eine didaktische Visualisierung geeignet, weil sie jeden relevanten Zustandswechsel explizit darstellen kann.

2.2. CPU-Scheduling in Betriebssystemen

CPU-Scheduling beschreibt die Auswahlstrategie zur Vergabe von Prozessorzeit an bereite Ausführungseinheiten. Ziel ist ein Ausgleich zwischen konkurrierenden Kriterien wie Reaktionszeit, Wartezeit, Durchlaufzeit, Auslastung und Durchsatz Stallings [2003].

In Mehrprogrammsystemen erfolgt diese Auswahl unter variierenden Ankunftszeiten und Laufzeiten. Scheduling-Verfahren werden daher üblicherweise als Heuristiken formuliert, deren Eignung vom Lastprofil und den Systemzielen abhängt. Zentrale Fragestellungen sind die Auswahl des nächsten Tasks, der Umgang mit Präemption und Kontextwechseln sowie die Bewertung über geeignete Kennzahlen.

2.2.1. Eingabeparameter und Entscheidungsgrundlage

Ein Scheduling-Szenario besteht im Kern aus einer Menge von Tasks. Für jeden Task werden mindestens eine Ankunftszeit und eine benötigte Ausführungszeit, die sogenannte Burst-Zeit, benötigt. Je nach Verfahren kommen eine Priorität, ein Zeitquantum oder eine

Zugehörigkeit zu einer Warteschlange hinzu.

Die Ankunftszeit legt fest, ab welchem Zeitpunkt ein Task berücksichtigt werden darf. Die Burst-Zeit beschreibt, wie viel CPU-Zeit bis zum Abschluss benötigt wird. Während der Simulation wird zusätzlich die verbleibende Ausführungszeit eines Tasks verfolgt.

Für jeden Zeitschritt wird zunächst bestimmt, welche Tasks angekommen und noch nicht abgeschlossen sind. Aus dieser Bereitschaftsmenge wählt der Scheduler anhand der jeweiligen Verfahrensregel einen Task. Anschließend wird dessen Restzeit reduziert und das entsprechende Ereignis beziehungsweise Segment im Ergebnisobjekt gespeichert. Diese Reihenfolge ist wichtig, weil ansonsten Ankunft, Präemption und Abschluss in einer mehrdeutigen Reihenfolge verarbeitet werden könnten.

2.2.2. Ziele und Zielkonflikte

Scheduling-Verfahren optimieren unterschiedliche Ziele, die nicht gleichzeitig maximal erfüllt werden können. Eine kurze durchschnittliche Wartezeit kann beispielsweise zu einer langen Wartezeit einzelner Tasks führen. Ein hoher Durchsatz kann mit einer schlechteren Reaktionszeit interaktiver Tasks einhergehen. Häufige Präemption kann die Reaktionsfähigkeit verbessern, erhöht aber die Anzahl der Kontextwechsel und damit den Verwaltungsaufwand.

Für die Bewertung wird deshalb kein einzelner universeller Gütewert vorausgesetzt. Stattdessen werden mehrere Kennzahlen gemeinsam betrachtet. Die Vergleichsansicht der Anwendung erlaubt zusätzlich, die Gewichtung dieser Kennzahlen an einen Anwendungsfall wie Interaktion, Batch-Verarbeitung oder Soft Real-Time anzupassen.

2.3. Präemptive und nicht-präemptive Scheduling-Verfahren

Scheduling-Verfahren lassen sich grundlegend in präemptive und nicht-präemptive Strategien einteilen. Diese Unterscheidung beeinflusst sowohl das Laufzeitverhalten als auch die Anzahl möglicher Kontextwechsel.

2.3.1. Nicht-präemptive Verfahren

Nicht-präemptive Verfahren erlauben einem Task, die CPU bis zum Abschluss oder bis zur Blockierung zu behalten. Der Scheduler unterbricht einen laufenden Task nicht aktiv, um einen anderen bereiten Task auszuführen. Entscheidungen treten dadurch in der Regel nur bei der Beendigung oder Blockierung des laufenden Tasks auf.

Vorteile nicht-präemptiver Verfahren sind eine einfache Ablaufstruktur und ein geringer Verwaltungsaufwand. Nachteilig sind potenziell lange Wartezeiten, wenn ein langer Task vor mehreren kürzeren Tasks ausgeführt wird. Typische Vertreter sind First-Come, First-

Served und nicht-präemptives Shortest Job First.

2.3.2. Präemptive Verfahren

Präemptive Verfahren erlauben dem Scheduler, einen laufenden Task aktiv zu unterbrechen und einen anderen Task einzuplanen. Auslöser sind typischerweise der Ablauf eines Zeitquantums, ein Timer-Interrupt oder die Ankunft eines höher priorisierten Tasks.

Dadurch steigen die Reaktionsfähigkeit und die Steuerbarkeit des Systems. Gleichzeitig nimmt jedoch die Anzahl der Kontextwechsel zu, wodurch zusätzlicher Verwaltungsaufwand entsteht Tanenbaum and Bos [2016].

2.4. Bewertungsmetriken

Die Bewertung eines Scheduling-Verfahrens benötigt messbare Größen, die aus dem Ablauf eines Simulationslaufs abgeleitet werden. Im Folgenden bezeichnet a_i die Ankunftszeit eines Tasks i , c_i seine Fertigstellungszeit, s_i den Zeitpunkt seiner ersten Ausführung und b_i seine gesamte benötigte CPU-Zeit.

2.4.1. Durchlaufzeit

Die Durchlaufzeit beschreibt die Zeitspanne vom Eintreffen bis zum Abschluss eines Tasks:

$$T_i = c_i - a_i \quad (2.1)$$

Sie umfasst sowohl die tatsächliche Ausführungszeit als auch Wartezeiten und Unterbrechungen. Die durchschnittliche Durchlaufzeit ergibt sich aus:

$$\bar{T} = \frac{1}{n} \sum_{i=1}^n T_i. \quad (2.2)$$

Eine geringe durchschnittliche Durchlaufzeit ist insbesondere für abgeschlossene Aufträge und Batch-Szenarien relevant. Sie beschreibt jedoch nicht, wie früh ein Task erstmals CPU-Zeit erhält.

2.4.2. Wartezeit

Die Wartezeit bezeichnet die Zeit, in der ein Task bereit ist, aber keine CPU erhält. Bei einem Task ohne Blockierung kann sie aus Durchlaufzeit und Burst-Zeit berechnet werden:

$$W_i = T_i - b_i. \quad (2.3)$$

Bei präemptiven Verfahren enthält diese Größe alle Zeitanteile zwischen den einzelnen Ausführungsabschnitten. Eine niedrige durchschnittliche Wartezeit kann anzeigen, dass die Bereitschaftsmenge insgesamt effizient bedient wurde. Zusätzlich ist die maximale Wartezeit relevant, weil ein guter Mittelwert lange Benachteiligungen einzelner Tasks verdecken kann.

2.4.3. Reaktionszeit

Die Reaktionszeit misst, wie lange ein Task nach seiner Ankunft bis zur ersten CPU-Zuteilung warten muss:

$$R_i = s_i - a_i. \quad (2.4)$$

Sie ist besonders für interaktive Systeme bedeutsam. Ein Verfahren kann eine akzeptable Durchlaufzeit erreichen, aber trotzdem eine schlechte Reaktionszeit aufweisen, wenn bestimmte Tasks erst spät erstmals ausgeführt werden.

2.4.4. Durchsatz und Kontextwechsel

Der Durchsatz beschreibt die Zahl abgeschlossener Tasks pro Zeiteinheit. Für einen Beobachtungszeitraum von t_0 bis t_1 kann er näherungsweise als Verhältnis der abgeschlossenen Tasks zur Zeitspanne angegeben werden:

$$X = \frac{N_{\text{fertig}}}{t_1 - t_0}. \quad (2.5)$$

Die Anzahl der Kontextwechsel ist keine reine Zeitmetrik, sondern ein Indikator für die Häufigkeit von Scheduling-Entscheidungen. Zusammen mit Präemptionsanzahl und Reaktionszeit hilft sie, den Zielkonflikt zwischen Responsivität und Verwaltungsaufwand zu beschreiben.

2.5. Scheduling-Verfahren

Die folgenden Verfahren werden im Visualizer als deterministische Strategien auf derselben Task- und Ereignisstruktur umgesetzt. Sie unterscheiden sich insbesondere darin, nach welcher Regel der nächste bereite Task ausgewählt wird und ob ein laufender Task durch einen anderen verdrängt werden darf.

Für die Simulation werden jedem Task mindestens eine Ankunftszeit und eine benötigte Ausführungszeit, die sogenannte Burst-Zeit, zugeordnet. Je nach Scheduling-Verfahren kommen zusätzliche Parameter wie Priorität, Zeitquantum oder eine Warteschlangenangehörigkeit hinzu. Diese Parameter werden in der Anwendung über das jeweilige Szenario

und die Einstellungen des Scheduling-Verfahrens festgelegt.

Die Verfahren werden im Folgenden zunächst fachlich beschrieben. Ihre konkreten Ablaufsequenzen und die daraus resultierenden Bewertungsmetriken werden in den entsprechenden Visualisierungen beziehungsweise im Ergebniskapitel dargestellt.

2.5.1. First-Come, First-Served und Shortest Job First als Referenz

First-Come, First-Served (FCFS) ordnet bereite Tasks nach ihrer Ankunft. Der zuerst angekommene Task wird zuerst ausgeführt. Das Verfahren ist einfach nachvollziehbar und benötigt keine komplexe Auswahlregel. Es kann jedoch zu langen Wartezeiten führen, wenn ein langer Task vor mehreren kurzen Tasks steht.

Shortest Job First (SJF) wählt den bereiten Task mit der kürzesten erwarteten Burst-Zeit. In der nicht-präemptiven Variante bleibt der ausgewählte Task bis zum Abschluss aktiv. Eine Auswahl nach der jeweils verbleibenden Ausführungszeit bezeichnet dagegen das präemptive Verfahren Shortest Remaining Time First (SRTF).

SJF dient in dieser Arbeit als Referenz für das Spannungsfeld zwischen durchschnittlicher Wartezeit und der notwendigen Kenntnis zukünftiger Ausführungszeiten. Die produktive Anwendung konzentriert sich dagegen auf präemptive Verfahren.

2.5.2. Round Robin

Round Robin verwendet eine zyklische Bereitschaftwarteschlange und ein festgelegtes Zeitquantum. Ein Task erhält höchstens ein Quantum. Wenn er danach noch nicht abgeschlossen ist, wird er am Ende der Warteschlange erneut eingereiht. Dadurch erhalten bereite Tasks wiederholt CPU-Zeit.

Ein kleines Quantum verbessert typischerweise die Reaktionsfähigkeit, erzeugt aber mehr Kontextwechsel. Ein großes Quantum reduziert den Verwaltungsaufwand und kann das Verhalten in Richtung FCFS verschieben.

Im Beispielszenario wird zunächst *P1* ausgeführt. Nach Ablauf des Quantums werden die bereiten Tasks entsprechend der Warteschlangenreihenfolge berücksichtigt. Die Visualisierung muss deshalb insbesondere den Quantumablauf, die Wiedereinreihung und den Wechsel des laufenden Tasks sichtbar machen.

2.5.3. Last-Come, First-Served

Bei Last-Come, First-Served (LCFS) wird der zuletzt eingetroffene bereite Task bevorzugt. In der für den Visualizer betrachteten präemptiven Variante kann ein neu eintreffender Task den aktuell laufenden Task verdrängen. Dadurch kann ein später eintreffender Task sehr schnell CPU-Zeit erhalten.

Gleichzeitig besteht die Gefahr, dass ältere Tasks wiederholt zurückgestellt werden und lange warten. Für die Visualisierung sind bei LCFS insbesondere die Ankunftszeitpunkte, die Präemptionsergebnisse und die Rückkehr verdrängter Tasks in die Bereitschaftsmenge relevant.

2.5.4. Strict Priority

Strict Priority wählt aus den bereiten Tasks denjenigen mit der höchsten Priorität. In der präemptiven Variante kann die Ankunft eines höher priorisierten Tasks den aktuell laufenden Task unmittelbar verdrängen. Die Prioritätskonvention muss dabei als Teil des Szenarios eindeutig definiert sein.

Die Stärke des Verfahrens liegt in der gezielten Bevorzugung wichtiger oder zeitkritischer Tasks. Die zentrale Schwäche ist das Risiko einer Starvation: Niedrig priorisierte Tasks können trotz bestehender Bereitschaft lange nicht ausgeführt werden. Eine Analyse sollte deshalb neben Mittelwerten auch maximale Wartezeiten und die Ereignisfolge betrachten.

2.5.5. Multilevel Feedback Queue

Die Multilevel Feedback Queue (MLFQ) verwendet mehrere Bereitschaftswarteschlangen mit unterschiedlichen Prioritätsstufen und Zeitquantumsgrößen. Neue Tasks werden entsprechend der gewählten Verfahrensvariante in eine Warteschlange eingeordnet. Verbraucht ein Task sein Quantum, ohne abzuschließen, kann er in eine niedrigere Warteschlange herabgestuft werden. Höhere Warteschlangen werden bevorzugt bedient.

Dadurch erhalten kurze oder interaktive Tasks häufig eine schnelle erste Reaktion. MLFQ ist adaptiv, weil die weitere Behandlung eines Tasks von seinem bisherigen Verhalten abhängt. Die Strategie kann dadurch unterschiedliche Task-Typen behandeln, ohne dass die vollständige Burst-Zeit vorher bekannt sein muss.

Für die konkrete Umsetzung des Visualizers müssen die Anzahl der Warteschlangen, die jeweiligen Quantumgrößen, die Regeln zur Herabstufung und die Behandlung neu eintreffender Tasks festgelegt werden. Aging oder periodische Promotions sind typische Gegenmaßnahmen gegen Starvation. Sie werden im fokussierten Simulationsmodell nur berücksichtigt, wenn sie ausdrücklich als Bestandteil der jeweiligen Verfahrensvariante definiert sind.

Im Visualizer wird MLFQ durch mehrere Ebenen, Segmentpositionen, Quantumabläufe und Herabstufungsergebnisse sichtbar gemacht. Dadurch lässt sich nicht nur das Endergebnis, sondern auch die Anpassung des Verfahrens an den bisherigen Task-Verlauf nachvollziehen.

2.6. Vergleich der theoretischen Perspektiven

Stallings sowie Tanenbaum und Bos behandeln beide Betriebssysteme, Prozesse und Scheduling, setzen jedoch unterschiedliche Akzente. Die Werke werden in dieser Arbeit nicht als widersprüchliche Theorien gegenübergestellt, sondern als sich ergänzende Zugänge verwendet.

Tabelle 2.2.: Ergänzende Perspektiven der verwendeten Betriebssystemliteratur.

Aspekt	Stallings	Tanenbaum und Bos
Grundperspektive	Systematische Betrachtung von Betriebssystemaufgaben, Ressourcen und Verwaltungszielen.	Konzeptionelle Betrachtung von Betriebssystemstrukturen, Prozessen, Threads und Abläufen.
Scheduling	Einordnung von Zielen und Kriterien der Prozessorzuteilung.	Erklärung von Prozessausführung, Präemption und dem Zusammenspiel der Betriebssystemkomponenten.
Nutzen für diese Arbeit	Begründung der Bewertungsmetriken und Zielkonflikte.	Begründung des Prozessmodells, der Zustandswechsel und der Ereignisorientierung.
Übertragung in den Visualizer	Vergleichsansicht mit Wartezeit, Durchlaufzeit, Reaktionszeit und Durchsatz.	Gantt-Segmente, Task-Zustände, Präemptionen und Kontextwechsel.

Stallings liefert damit vor allem den systematischen Rahmen für die Frage, nach welchen Kriterien Scheduling-Verfahren bewertet werden können. Tanenbaum und Bos ergänzen diesen Rahmen um die Perspektive auf Prozesse und die Abläufe, durch die Scheduling-Entscheidungen wirksam werden.

Die Anwendung benötigt beide Ebenen: Ohne Metriken wären Scheduling-Verfahren nicht vergleichbar; ohne Zustands- und Ereignismodell bliebe unklar, wie ein Ergebnis zustande gekommen ist Stallings [2003], Tanenbaum and Bos [2016].

2.7. Algorithmusvisualisierung als Vermittlungsform

Eine Scheduling-Visualisierung übersetzt abstrakte Zustandsänderungen in eine zeitlich beobachtbare Darstellung. Für die vorliegende Anwendung bedeutet dies, dass ein Task nicht nur als fertige Kennzahl, sondern als Folge von Ankunft, Bereitschaft, Ausführung, Präemption und Abschluss sichtbar wird.

Die Gantt-Ansicht stellt die zeitliche Ordnung her, während Status- und Ereignisansichten die Bedeutung einzelner Abschnitte ergänzen. Auf diese Weise können sowohl das Ergebnis eines Scheduling-Verfahrens als auch der Weg dorthin untersucht werden.

Algorithmusvisualisierung ist dabei nicht automatisch mit Lernwirksamkeit gleichzusetzen. Die Meta-Studie von Hundhausen, Douglas und Stasko zeigt, dass der Nutzen algorithmischer Visualisierungen von der Gestaltung, der Aufgabenstellung und der aktiven Auseinandersetzung mit der Darstellung abhängt Hundhausen et al. [2002].

Für diese Arbeit folgt daraus die Anforderung, Entscheidungswege und Interaktionsmöglichkeiten bereitzustellen. Eine empirische Untersuchung, ob Studierende dadurch tatsächlich besser lernen, ist ausdrücklich nicht Gegenstand dieser Arbeit.

2.8. Accessibility als fachliche und gestalterische Grundlage

Die fachliche Komplexität des Scheduling-Modells darf nicht dazu führen, dass zentrale Zustände ausschließlich visuell oder nur mit einer Maus zugänglich sind. Für die Anwendung sind daher eine semantische Struktur, Tastaturbedienung, ein sichtbarer Fokus, verständliche Statusmeldungen und eine nicht ausschließlich farbliche Unterscheidung relevant.

Die Web Content Accessibility Guidelines 2.1 dienen hierfür als normative Referenz W3C [2018]. Heinecke und Gerken ergänzen diese normative Perspektive um Grundlagen der Mensch-Computer-Interaktion und der Gestaltung verständlicher Interaktionen Heinecke and Gerken [2024].

Accessibility wird in dieser Arbeit nicht als nachträgliche Prüfung einzelner Farben verstanden. Sie betrifft auch die Modellierung der Zustände: Ein Dialog benötigt Fokusmanagement, ein Zeitregler eine verständliche Beschriftung und ein Gantt-Segment einen zugänglichen Namen. Diese Grundlagen führen direkt zu den in Kapitel 3 operationalisierten Anforderungen A11Y-01 bis A11Y-08.

2.9. Zwischenfazit

Dieses Kapitel hat das fachliche Modell für die folgenden Teile der Arbeit hergeleitet. Prozesse und Threads bilden den theoretischen Rahmen, während Tasks im Visualizer als zustandsbehaftete, schedulierbare Ausführungseinheiten modelliert werden.

Präemption, Kontextwechsel und Ankunftsereignisse erzeugen eine zeitliche Folge, die über Kennzahlen und Visualisierung analysiert werden kann. Dabei wurde außerdem herausgearbeitet, dass ein Einkernprozessor zu jedem Zeitpunkt nur einen Thread tatsächlich ausführen kann. Die scheinbare Gleichzeitigkeit mehrerer Programme entsteht dort durch schnelle Wechsel der Prozessorzuteilung. Auf Mehrkernprozessoren ist dagegen eine echte parallele Ausführung mehrerer Threads oder Prozesse möglich.

Für den Systementwurf ergeben sich daraus drei Konsequenzen. Erstens müssen Task-Zustände, Ereignisse und Segmente auf einer gemeinsamen Datenbasis beruhen. Zweitens müssen mehrere Metriken parallel betrachtet werden, weil Scheduling-Ziele in Konkurrenz stehen. Drittens muss die Interaktion den Ablauf erklärbar und zugänglich machen. Diese Konsequenzen bilden die Brücke von den Grundlagen zur Anforderungsanalyse und zum Systementwurf.

3. Anforderungsanalyse

Die Anforderungen ergeben sich aus der Verbindung der fachlichen Grundlagen des CPU-Schedulings mit didaktischen Zielen und softwaretechnischen Qualitätsansprüchen. Ziel ist eine Anwendung, die ausgewählte präemptive Scheduling-Verfahren fachlich korrekt simuliert und ihre Abläufe für Lernende verständlich und zugänglich darstellt.

Die Anforderungen werden in fachliche, didaktische, interaktionsbezogene, technische und auf Accessibility bezogene Anforderungen gegliedert. Die daraus abgeleiteten Anforderungen bilden die Grundlage für den Systementwurf und die spätere Qualitätssicherung.

3.1. Zielgruppe und Einsatzkontext

Die primäre Zielgruppe sind Studierende in einführenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung soll sowohl die begleitete Demonstration im Unterricht als auch die eigenständige Analyse von Scheduling-Szenarien unterstützen.

Der Visualizer ist als ergänzendes Lehr- und Lernmittel vorgesehen. Er ersetzt keine theoretische Einführung in Betriebssysteme und führt keine realen Betriebssystemprozesse aus. Stattdessen modelliert er die zeitliche CPU-Zuteilung ausgewählter Scheduling-Verfahren in einer kontrollierten Simulationsumgebung.

Die Anwendung soll insbesondere dabei unterstützen, die Auswirkungen verschiedener Scheduling-Regeln auf die Reihenfolge der Ausführung, Präemptionen, Wartezeiten, Reaktionszeiten und Kontextwechsel nachzuvollziehen.

3.2. Fachliche Anforderungen

Die Anwendung muss die präemptiven Scheduling-Verfahren Round Robin, LCFS, Strict Priority und Multilevel Feedback Queue (MLFQ) simulieren. Die Verfahren müssen auf derselben grundlegenden Task- und Ereignisstruktur arbeiten, damit ihre Abläufe und Kennzahlen miteinander verglichen werden können.

Die Anwendung unterstützt neben der direkten Bearbeitung auch die persistente Speicherung sowie den JSON-basierten Import und Export von Szenarien. Ein Szenario kann zusammen mit den zugehörigen Simulationsläufen exportiert werden. Beim Import werden sowohl die Task-Daten und Verfahrensparameter als auch die gespeicherten Runs wieder-

hergestellt.

Im Simulationsmodell bezeichnet der Begriff *Task* eine abstrahierte schedulierbare Ausführungseinheit. Ein Task besitzt mindestens eine eindeutige Kennung, eine Ankunftszeit, eine benötigte Ausführungszeit und eine Priorität. Zusätzlich können eine Bezeichnung, eine Farbe und eine Gruppenzugehörigkeit hinterlegt werden.

Die Anwendung muss folgende fachliche Anforderungen erfüllen:

- **FR-01:** Es müssen Szenarien mit mehreren Tasks erstellt, bearbeitet und gelöscht werden können.
- **FR-02:** Für jeden Task müssen mindestens eine Task-ID, eine Ankunftszeit, eine Burst-Zeit und eine Priorität festgelegt werden können.
- **FR-03:** Die Anwendung muss die präemptiven Verfahren Round Robin, LCFS, Strict Priority und MLFQ simulieren.
- **FR-04:** Die Auswahl eines Tasks darf ausschließlich auf Grundlage der zum jeweiligen Zeitpunkt bereiten und noch nicht abgeschlossenen Tasks erfolgen.
- **FR-05:** Präemptionen, Kontextwechsel, Zustandsänderungen und Fertigstellungen müssen im Simulationsablauf erfasst werden.
- **FR-06:** Simulationslogik, Ereignisverlauf, Zeitsegmente und Kennzahlen müssen aus derselben Simulationsgrundlage abgeleitet werden.
- **FR-07:** Bei identischen Eingabedaten und identischen Verfahrensparametern muss die Simulation deterministisch denselben Ablauf erzeugen.

Die Simulation verwendet für die vier Verfahren folgende verfahrensspezifische Parameter und Regeln.

3.2.1. Round Robin

Round Robin verwendet eine FIFO-Bereitschaftswarteschlange und ein globales Zeitquantum. Das Zeitquantum muss für die Simulation konfigurierbar sein.

Läuft das Quantum eines noch nicht abgeschlossenen Tasks ab und befindet sich mindestens ein weiterer Task in der Bereitschaftsmenge, wird der laufende Task präemptiert und am Ende der Bereitschaftswarteschlange erneut eingereiht. Ist kein weiterer Task bereit, wird das Quantum zurückgesetzt und der aktuelle Task weiter ausgeführt.

3.2.2. LCFS

LCFS wird in der Anwendung präemptiv umgesetzt. Der zuletzt in die Bereitschaftsmenge eingereihte Task wird bevorzugt ausgewählt. Trifft während der Ausführung ein neuer Task ein, kann dieser den aktuell laufenden Task verdrängen.

Für gleichzeitig eintreffende Tasks kann die Reihenfolge über eine konfigurierbare Tie-Break-Regel bestimmt werden. Unterstützt werden die Stack-Reihenfolge und die Task-ID.

3.2.3. Strict Priority

Strict Priority wird präemptiv umgesetzt. Aus den bereiten Tasks wird zunächst der Task mit der höchsten Priorität ausgewählt. In der Anwendung entspricht ein kleinerer Prioritätswert einer höheren Priorität.

Besitzt ein neu eingetroffener Task eine höhere Priorität als der aktuell laufende Task, wird dieser verdrängt. Bei gleicher Priorität kann eine Tie-Break-Regel ausgewählt werden. Die Anwendung unterstützt folgende Tie-Break-Regeln:

- längste Wartezeit,
- kürzeste Restzeit,
- FIFO-Reihenfolge,
- früheste Ankunftszeit,
- Task-ID.

3.2.4. Multilevel Feedback Queue

Die Multilevel Feedback Queue verwendet mehrere Bereitschaftswarteschlangen mit unterschiedlichen Prioritätsstufen. Die Anzahl der Queue-Stufen und das Basisquantum müssen konfigurierbar sein.

Neue Tasks starten in der höchsten Queue-Stufe. Die Queue-Stufe wird durch einen numerischen Level beschrieben, wobei kleinere Werte eine höhere Priorität darstellen. Die Quantumgröße einer Queue-Stufe ergibt sich aus dem Basisquantum und der aktuellen Queue-Stufe:

$$q_l = q_{\text{Basis}} \cdot (l + 1), \quad (3.1)$$

wobei l den Queue-Level bezeichnet. Bei einem Basisquantum von 2 ergeben sich beispielsweise die Quantumgrößen 2, 4 und 6 für die Queue-Level 0, 1 und 2.

Verbraucht ein Task sein Quantum, ohne abgeschlossen zu sein, wird er um eine Queue-Stufe herabgestuft. Ein laufender Task wird präemptiert, sobald ein bereiter Task aus einer höher priorisierten Queue vorhanden ist. Eine automatische Beförderung durch Aging ist im fokussierten Simulationsmodell nicht vorgesehen.

3.3. Bereitstellung von Simulationsergebnissen

Die Anwendung muss die Ergebnisse eines Simulationslaufs in mehreren miteinander verbundenen Darstellungen bereitstellen.

- **FR-08:** Der zeitliche Ablauf muss in einer Gantt-Ansicht dargestellt werden.
- **FR-09:** Relevante Ereignisse wie Ankunft, Dispatch, Start, Präemption, Quantumablauf, Kontextwechsel und Abschluss müssen in einer Ereignisliste nachvollziehbar sein.
- **FR-10:** Die aktuellen Zustände der Tasks müssen während des Ablaufs sichtbar sein.
- **FR-11:** Die Anwendung muss zentrale Kennzahlen pro Simulationslauf berechnen und anzeigen.
- **FR-12:** Ein Szenario muss zusammen mit den zugehörigen Simulationsläufen in einem JSON-basierten Format exportiert werden können.
- **FR-13:** Exportierte Szenarien einschließlich ihrer Simulationsläufe müssen wieder importiert werden können.

Für einen Simulationslauf werden insbesondere folgende Kennzahlen bereitgestellt:

- durchschnittliche Wartezeit,
- durchschnittliche Durchlaufzeit,
- durchschnittliche Reaktionszeit,
- maximale Wartezeit,
- Anzahl der Präemptionen,
- Anzahl der Kontextwechsel.

Die Reaktionszeit beschreibt dabei die Zeitspanne zwischen der Ankunft eines Tasks und seiner ersten CPU-Zuteilung. Die Präemptionsanzahl umfasst sowohl explizite Verdrängungen als auch Quantumabläufe, bei denen ein noch nicht abgeschlossener Task die CPU abgeben muss.

Ein Kontextwechsel bezeichnet in dieser Arbeit den Wechsel der CPU-Zuteilung von einem zuvor laufenden Task zu einem anderen Task. Der initiale Start des ersten Tasks sowie der Start eines Tasks nach einer Leerlaufphase werden nicht als Kontextwechsel gezählt. Die Fortsetzung desselben Tasks wird ebenfalls nicht als Kontextwechsel gewertet.

3.4. Didaktische Anforderungen

Die Anwendung soll nicht nur Endwerte liefern, sondern auch die Entscheidungswege des Schedulers sichtbar machen. Lernende sollen erkennen können, warum ein Task ausgewählt, unterbrochen, erneut eingeplant oder zurückgestellt wird.

Im Zentrum stehen folgende didaktische Anforderungen:

- **DID-01 – Transparenz:** Dispatch, Präemption, Kontextwechsel, Quantumablauf und Abschluss müssen eindeutig dargestellt werden.
- **DID-02 – Vergleichbarkeit:** Mehrere Scheduling-Verfahren müssen auf einer identischen Szenariobasis gegenübergestellt werden können.
- **DID-03 – Kontrollierbares Lerntempo:** Die Anwendung muss sowohl einen Schrittmodus als auch eine kontinuierliche Wiedergabe unterstützen.
- **DID-04 – Nachvollziehbarkeit:** Relevante Zustandsänderungen müssen mit verständlichen Ereignisinformationen erklärt werden.
- **DID-05 – Mehrperspektivität:** Zeitlicher Ablauf, Task-Zustände und Bewertungsmetriken müssen miteinander in Beziehung gesetzt werden können.

Die Anwendung soll dadurch das Verständnis der Auswahlregeln und ihrer Auswirkungen auf Wartezeit, Reaktionszeit, Durchlaufzeit und Kontextwechsel unterstützen. Eine empirische Untersuchung des tatsächlichen Lernerfolgs ist nicht Gegenstand dieser Arbeit.

3.5. Interaktions- und Usability-Anforderungen

Die Oberfläche soll die kognitive Belastung reduzieren und den typischen Arbeitsablauf unterstützen:

1. Szenario definieren,
2. Scheduling-Verfahren auswählen,
3. verfahrensspezifische Parameter festlegen,
4. Simulation starten,
5. Ablauf untersuchen,
6. Ergebnisse interpretieren,
7. Verfahren miteinander vergleichen.

Daraus ergeben sich folgende Anforderungen:

- **UX-01:** Tasks müssen angelegt, bearbeitet und gelöscht werden können.
- **UX-02:** Szenarien müssen gespeichert und erneut geladen werden können.
- **UX-03:** Eine Simulation muss gestartet, pausiert, fortgesetzt und zurückgesetzt werden können.
- **UX-04:** Der Simulationsablauf muss schrittweise durchlaufen werden können.
- **UX-05:** Die Anwendung muss Zeitsprünge beziehungsweise eine zeitbasierte Ablaufsteuerung unterstützen.

- **UX-06:** Der aktuell laufende Task und relevante Zustandsänderungen müssen unmittelbar erkennbar sein.
- **UX-07:** Die wesentlichen Bedienfunktionen müssen in einer konsistenten und nachvollziehbaren Reihenfolge erreichbar sein.
- **UX-08:** Ungültige oder fehlende Eingaben müssen verständlich angezeigt werden.
- **UX-09:** Die Bedienlogik darf die fachliche Analyse nicht durch unnötige Interaktionsschritte überlagern.

Szenarien und die dazugehörigen Simulationsläufe müssen als JSON-Datei exportiert und zu einem späteren Zeitpunkt wieder importiert werden können. Der Export dient dadurch nicht nur der Speicherung der Eingabedaten, sondern auch der Sicherung und Weitergabe bereits durchgeführter Simulationen.

3.6. Vergleich mehrerer Simulationsläufe

Die Anwendung muss mehrere Läufe auf einer identischen Szenariobasis miteinander vergleichen können. Hierzu müssen mehrere Algorithmen auf dasselbe Task-Szenario angewendet werden können.

Die Vergleichsübersicht soll neben den einzelnen Kennzahlen auch eine Bewertungsmatrix bereitstellen. Dafür können unterschiedliche Anwendungsfälle ausgewählt werden, deren Gewichtung die Bedeutung einzelner Kennzahlen verändert. Vorgesehen sind:

- **Interaktiv / UI:** Schwerpunkt auf Reaktionszeit und kurzen Wartezeiten.
- **Batch / HPC:** Schwerpunkt auf Durchsatz und Durchlaufzeit; Kontextwechsel werden als kostenintensiv betrachtet.
- **Webserver:** Schwerpunkt auf kurzen Wartezeiten und stabiler Reaktionszeit.
- **Soft Real-Time:** Schwerpunkt auf Reaktionszeit und Präemptionsverhalten.
- **Eigener Anwendungsfall:** Individuelle Gewichtung der verfügbaren Kennzahlen.

Die Anwendung soll aus den gewichteten Kennzahlen einen vergleichenden Score ableiten und den besten sowie den schwächsten Lauf kenntlich machen. Der Score dient dabei als anwendungsbezogene Vergleichshilfe und nicht als allgemeingültige Bewertung eines Scheduling-Verfahrens.

3.7. Anforderungen an Accessibility

Da die Anwendung im Lehrkontext eingesetzt wird, sind grundlegende Anforderungen an die Barrierefreiheit zu berücksichtigen. Dazu gehören eine semantisch klare Struktur, eine nachvollziehbare Fokusführung, verständliche Statuskommunikation und ausreichende

Kontraste.

Ziel ist, dass zentrale Lern- und Bedienfunktionen auch ohne ausschließliche visuelle Orientierung und ohne Maus nutzbar bleiben W3C [2018]. Die folgende Operationalisierung begrenzt den Prüfstand auf die tatsächlich relevante Scheduler-Oberfläche. Sie stellt keine vollständige Konformitätserklärung für alle WCAG-Erfolgskriterien dar.

Tabelle 3.1.: Operationalisierte Accessibility-Anforderungen des fokussierten Prüfstands.

ID	Anforderung	Referenz		Prüfverfahren und Grenze
A11Y-01	Semantische Struktur und verständliche Namen für Dialoge, Steuerelemente und Ansichten.	WCAG 4.1.2	1.3.1,	Statische und manuelle Prüfung; keine vollständige Assistive-Tech-Kompatibilitätsmessung.
A11Y-02	Zentrale Funktionen sind vollständig per Tastatur bedienbar.	WCAG 2.1.1		Keyboard-Walkthrough; komplexe Browser-/Assistive-Tech-Kombinationen bleiben offen.
A11Y-03	Der Fokus ist sichtbar und logisch geführt. In modalen Dialogen bleibt er innerhalb des Dialogs und wird nach dem Schließen an das auslösende Element zurückgegeben.	WCAG 2.4.7	2.4.3,	Codeprüfung und manueller Dialogtest; keine formale Fokusabdeckungsmessung.
A11Y-04	Zustandsänderungen von Playback und Vergleich werden verständlich angekündigt.	WCAG 4.1.3		Prüfung der Statuskanäle und manueller Test; Sprachverständlichkeit wird nicht empirisch bewertet.
A11Y-05	Gantt-SVG und interaktive Segmente besitzen beschreibende Namen und Tastaturzugriff.	WCAG 2.1.1	1.1.1,	Statische Prüfung und manuelle Segmentprüfung; SVG wird nicht mit jedem Screenreader getestet.
A11Y-06	Farbe ist nicht alleiniger Informationsträger und Fokus-/Textkontraste sind ausreichend.	WCAG 1.4.3, 1.4.11	1.4.1,	Kontrastprüfung und manuelle Farbnutzung; frei kombinierbare Farben bleiben eine Grenze.
A11Y-07	Die Oberfläche bleibt bei Vergrößerung und schmalem Viewport nutzbar.	WCAG 1.4.10	1.4.4,	Prüfung bei 200 % und schmalem Viewport; keine Prüfung aller Endgeräte.
A11Y-08	Nicht notwendige Bewegung wird bei reduzierter Bewegungspräferenz vermieden.	WCAG 2.3.3		Prüfung der Medienpräferenz und des Playbacks; keine Aussage zu vestibulären Wirkungen.

WCAG 2.1 bildet den normativen Maßstab. Das im Projektkontext verwendete Kurs- oder Handoutmaterial dient als angewandte Entwurfsgrundlage, ersetzt jedoch weder die Primärquelle noch die hier vorgenommene Prüfung.

3.8. Technische Qualitätsziele

Die Anwendung soll modular, deterministisch, testbar und erweiterbar aufgebaut sein. Daraus ergeben sich folgende nicht-funktionale Anforderungen:

- **NFR-01 – Determinismus:** Bei identischer Eingabe und identischer Konfiguration muss die Simulation denselben Ereignis- und Ergebnisverlauf erzeugen.
- **NFR-02 – Trennung der Verantwortlichkeiten:** Simulationskern, Zustandsverwaltung und Darstellung müssen logisch voneinander getrennt sein.
- **NFR-03 – Testbarkeit:** Die Simulationslogik und zentrale Interaktionspfade müssen unabhängig voneinander überprüfbar sein.
- **NFR-04 – Erweiterbarkeit:** Weitere Scheduling-Verfahren und Darstellungsformen sollen über klar definierte Schnittstellen ergänzt werden können.
- **NFR-05 – Konsistenz:** Visualisierung, Ereignisprotokoll und Bewertungsmetriken müssen auf derselben Simulationsausgabe beruhen.
- **NFR-06 – Wartbarkeit:** Änderungen an einem Scheduling-Verfahren sollen möglichst ohne Änderungen an den Darstellungs- und Vergleichskomponenten vorgenommen werden können.
- **NFR-07 – Bedienbarkeit:** Die zentrale Scheduler-Oberfläche soll ohne unnötige Bedienghürden nutzbar sein.

Diese Qualitätsziele sichern sowohl die Wartbarkeit der Anwendung als auch ihre fachliche Verlässlichkeit im Lehrbetrieb.

Die folgenden Listen fassen die zuvor hergeleiteten Anforderungen für die weitere Rückverfolgbarkeit im Systementwurf und in der Qualitätssicherung zusammen.

3.9. Abgeleitete funktionale und nicht-funktionale Anforderungen

Die Anforderungen lassen sich abschließend in funktionale und nicht-funktionale Kernanforderungen zusammenfassen.

3.9.1. Funktionale Anforderungen

- **FR-01:** Erstellen, Bearbeiten und Löschen von Szenarien und Tasks.
- **FR-02:** Erfassung von Task-ID, Ankunftszeit, Burst-Zeit und Priorität.
- **FR-03:** Simulation von Round Robin, LCFS, Strict Priority und MLFQ.
- **FR-04:** Konfiguration verfahrensspezifischer Parameter wie Zeitquantum, Tie-Break-

Regel, Queue-Stufen und Basisquantum.

- **FR-05:** Visualisierung des zeitlichen Ablaufs einschließlich Präemptionen und Zustandswechseln.
- **FR-06:** Anzeige der Ereignisse eines Simulationslaufs.
- **FR-07:** Anzeige der zentralen Bewertungsmetriken.
- **FR-08:** Vergleich mehrerer Läufe auf identischer Szenariobasis.
- **FR-09:** Gewichteter Vergleich anhand verschiedener Anwendungsfälle.
- **FR-10:** Interaktive Ablaufsteuerung mit Start, Pause, Fortsetzen, Schrittmodus, Zeitsprung und Zurücksetzen.
- **FR-11:** Speichern und Laden von Szenarien innerhalb der Anwendung.
- **FR-12:** Export eines Szenarios einschließlich der zugehörigen Simulationsläufe als JSON-Datei.
- **FR-13:** Import zuvor exportierter Szenarien einschließlich ihrer Simulationsläufe.

3.9.2. Nicht-funktionale Anforderungen

- **NFR-01:** Deterministisches Verhalten bei identischer Eingabe und Konfiguration.
- **NFR-02:** Klare Trennung von Simulationslogik, Zustandsverwaltung und Darstellung.
- **NFR-03:** Verständliche und konsistente Benutzerführung.
- **NFR-04:** Berücksichtigung der festgelegten Accessibility-Anforderungen.
- **NFR-05:** Erweiterbarkeit für zusätzliche Verfahren und Darstellungsformen.
- **NFR-06:** Testbarkeit der Simulationslogik und zentraler Interaktionspfade.
- **NFR-07:** Konsistenz zwischen Simulationsablauf, Ereignisprotokoll, Visualisierung und Kennzahlen.

Die abgeleiteten Anforderungen bilden die Grundlage für den Systementwurf im folgenden Kapitel. Die dort beschriebenen Architektur- und Interaktionsentscheidungen werden auf diese Anforderungen zurückgeführt und anschließend im Rahmen der Implementierung und Qualitätssicherung überprüft.

4. Systementwurf

Der Systementwurf folgt der Leitidee, fachliche Korrektheit und verständliche Interaktion gemeinsam umzusetzen. Ein Scheduling-Visualizer soll nicht nur Ergebnisse berechnen, sondern auch Entscheidungswege transparent machen. Entsprechend verbindet dieses Kapitel Architekturentscheidungen mit didaktischen Anforderungen.

4.1. Entwurfsziele und Leitprinzipien

Aus der Anforderungsanalyse ergeben sich vier zentrale Entwurfsziele:

- **Fachliche Konsistenz:** Simulation, Ereignisverlauf und Kennzahlen basieren auf einer gemeinsamen Zustandsbasis.
- **Didaktische Nachvollziehbarkeit:** Zustandswechsel und Präemptionen sind zeitlich eindeutig darzustellen.
- **Deterministik:** Identische Eingaben führen zu identischen Abläufen.
- **Erweiterbarkeit:** Neue Algorithmen und Darstellungsformen sollen ohne grundlegenden Umbau integrierbar sein.

Diese Ziele werden durch eine komponentenbasierte Struktur mit klaren Verantwortlichkeiten umgesetzt. Simulationslogik und Darstellung bleiben bewusst entkoppelt, sodass fachliche Änderungen kontrolliert möglich sind und die Oberfläche als Vermittlungsschicht wirken kann.

4.2. Gesamtarchitektur und Schichtenmodell

Das System ist in mehrere logisch getrennte, miteinander verbundene Bereiche gegliedert:

- **Präsentationsschicht:** Darstellung der Fokusansicht, der Vergleichsübersicht, der Gantt-Zeitachse, der Statusansicht und der Bewertungsmatrix.
- **Anwendungslogik:** Wiederverwendbare Composables für Szenarioverwaltung, Vergleich und Wiedergabesteuerung.
- **Simulationskern:** Deterministische Berechnung des zeitlichen Ablaufs durch die Scheduling Engine.

- **Domänenmodell:** Gemeinsame fachliche Datenbasis aus Szenario und Simulation-Run.

Die konkrete Komponenten- und Datenstruktur ist in Abbildung 4.1 dargestellt. Die Anwendungshülle `AppShell.vue` vermittelt zwischen Benutzerinteraktion, Composables und den sichtbaren Ansichten. Die Steuerung ist damit nicht in einer eigenständigen fachlichen Zustandswelt gekapselt, sondern wirkt über die Anwendungslogik auf die gemeinsame Datenbasis ein.

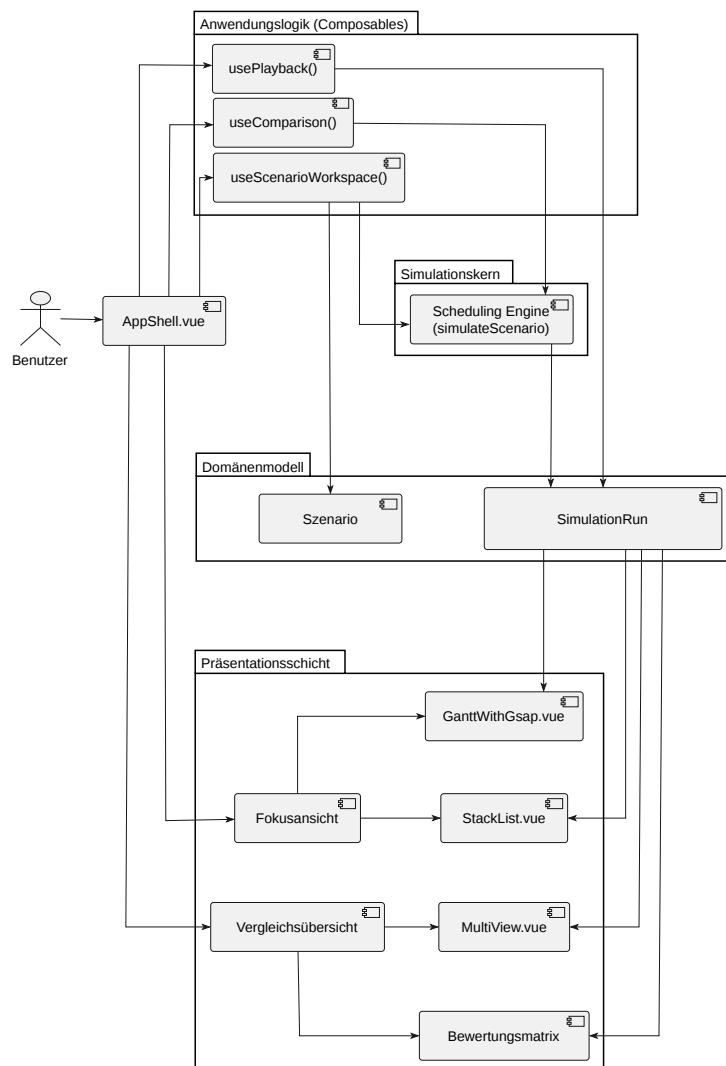


Abbildung 4.1.: Komponentenmodell der Anwendung mit Präsentationsschicht, Composables, Simulationskern und Domänenmodell.

Abbildung 4.1 zeigt den zentralen Daten- und Kontrollfluss der Anwendung. Die Benut-

zerinteraktion wird zunächst von `AppShell.vue` aufgenommen und an die zuständigen Composables weitergeleitet. `useScenarioWorkspace()` verwaltet Szenarien und stößt die Berechnung eines Laufs an, während `usePlayback()` die Navigation durch die erzeugten Zustandsabbilder steuert und `useComparison()` den Vergleich mehrerer Läufe unterstützt.

Die Scheduling Engine mit `simulateScenario()` verarbeitet das Szenario und erzeugt ein `SimulationRun`. Dieses Ergebnisobjekt bildet die gemeinsame fachliche Grundlage für die Präsentationsschicht. Die Fokusansicht nutzt daraus unter anderem die Gantt-Darstellung durch `GanttWithGsap.vue` und die Statusdarstellung durch `StackList.vue`. Die Vergleichsübersicht verwendet `MultiView.vue` und die Bewertungsmatrix, um mehrere erzeugte Läufe gegenüberzustellen.

Die Architektur trennt damit die Berechnung eines Ablaufs von seiner Darstellung, ohne den Datenfluss unnötig zu vervielfachen. Neue Ansichten können auf den bestehenden Ergebnisstrukturen aufbauen, während Änderungen an der Scheduling Engine nicht unmittelbar eine Anpassung jeder einzelnen Präsentationskomponente erfordern.

4.3. Komponenten und Verantwortlichkeiten

4.3.1. Szenario- und Konfigurationskomponente

Diese Komponente verwaltet die Eingabedaten der Prozesse (z. B. Ankunftszeit, Burst-Zeit, Priorität) sowie algorithmusspezifische Parameter (z. B. Zeitquantum). Zusätzlich übernimmt sie die Validierung und Normalisierung der Eingaben, sodass alle Läufe auf strukturell konsistenten Daten aufsetzen.

4.3.2. Simulationskomponente

Die Simulationskomponente bildet den fachlichen Kern. Aus Szenario und Verfahrensdefinition erzeugt sie den vollständigen Ablauf als deterministische Folge von Zustandsübergängen. Ergebnisse werden nicht nur als Endzustand abgelegt, sondern in auswertbaren Strukturen bereitgestellt (Segmente, Ereignisse, Snapshots, Kennzahlen).

4.3.3. Visualisierungs- und Interaktionskomponente

Diese Komponente überführt Simulationsdaten in Analyseansichten: zeitlicher Ablauf (Gantt), Warteschlangen-/Statusdarstellung und Kennzahlen. Gleichzeitig steuert sie die Wiedergabe über Interaktionsfunktionen und ermöglicht den Vergleich mehrerer Läufe auf identischer Datenbasis.

4.4. Datenfluss und Zustandsmodell

Der Datenfluss folgt einem festen Muster:

1. Eingabeszenario wird erfasst und validiert.
2. Der Simulationskern berechnet den Ablauf für ein Verfahren.
3. Ergebnisstrukturen (Ereignisse, Segmente, Snapshots, Metriken) werden bereitgestellt.
4. Die Darstellungsschicht rendert die aktuelle Sicht und reagiert auf Interaktionsbefehle.

Entscheidend ist, dass Visualisierung und Kennzahlen auf denselben Ergebnisstrukturen basieren. Dadurch entstehen keine widersprüchlichen Anzeigen zwischen Ablaufdarstellung und numerischer Bewertung.

4.5. Interaktionsmodell für den Lernprozess

Das Interaktionsmodell folgt einem didaktisch motivierten Nutzungsfluss:

1. Szenario definieren,
2. Verfahren auswählen,
3. Ablauf steuern und beobachten,
4. Ergebnisse interpretieren,
5. Verfahren vergleichen.

Die Interaktion ist damit kein rein technischer Bedienvorgang, sondern Teil des Lernprozesses. Die Steuerung beeinflusst die Sicht auf den Ablauf (z. B. Schrittmodus), ohne die fachliche Berechnung selbst zu verändern.

4.6. HCI-Leitlinien im Entwurf

Für eine Lernanwendung ist die Benutzeroberfläche ein fachlich relevantes Element. Der Entwurf folgt daher grundlegenden HCI-Leitlinien:

- zentrale Zustände müssen direkt sichtbar sein,
- wichtige Ereignisse (Präemption, Kontextwechsel, Abschluss) müssen klar markiert werden,
- Bedienung und Analyse sollen in einem konsistenten Ablauf zusammenwirken,
- die Oberfläche soll Orientierung bieten und zusätzliche kognitive Last minimieren.

Damit wird sichergestellt, dass die Anwendung nicht nur technisch korrekt arbeitet, sondern auch als didaktisches Werkzeug nutzbar ist.

4.7. Accessibility als Entwurfsprinzip

Accessibility wird im Entwurf als Querschnittseigenschaft der Interaktions- und Darstellungsschicht behandelt. Dialoge besitzen einen eindeutig benannten Zustand, einen initialen Fokus und eine definierte Rückgabe an das auslösende Element. Das Composable `useFocusTrap` kapselt diese Logik, sodass Welcome-, Auswahl-, Generator- und Ranking-Dialog nach demselben Muster bedient werden.

Die Gantt-Ansicht wird als strukturiertes SVG mit Titel, Beschreibung und Zusammenfassung ausgegeben. Interaktive Segmente erhalten zusätzlich eigene zugängliche Namen und können per Enter oder Leertaste aktiviert werden. Die Vergleichsansicht verwendet eine echte Tabstruktur mit ausgewähltem Tab, zugehörigem Panel und Pfeiltastennavigation. Statusänderungen werden über separate Statuskanäle angekündigt, damit sie nicht ausschließlich über Farbe oder Bewegung vermittelt werden.

Farbe dient der visuellen Orientierung, ist jedoch nicht die einzige Unterscheidung. Aktive, beste und schwächste Zustände erhalten zusätzlich Text oder strukturelle Merkmale. Fokusindikatoren, Forced-Colors-Regeln, Beschriftungen für Zeitsteuerungen und die Reaktion auf reduzierte Bewegung sind Teil des Entwurfs. Diese Entscheidungen adressieren die A11Y-Anforderungen aus Tabelle 3.1; sie begründen jedoch keine vollständige WCAG-Konformität.

4.8. Erweiterbarkeit und Entwurfsgrenzen

Die Architektur ist auf Erweiterbarkeit ausgelegt: Neue Scheduling-Verfahren können als zusätzliche Strategien im Simulationskern ergänzt werden, ohne die UI grundlegend umzubauen. Ebenso lassen sich neue Kennzahlen oder Ansichten integrieren, sofern sie auf den vorhandenen Ergebnisstrukturen aufsetzen.

Eine bewusste Entwurfsgrenze liegt in der Fokussierung auf präemptive Verfahren und den Lehrkontext. Die Architektur ist zwar übertragbar, aber nicht als vollständiger Ersatz für produktive Betriebssystem-Scheduler konzipiert.

5. Implementierung

Die Implementierung überführt den in Kapitel 4 beschriebenen Entwurf in eine lauffähige Webanwendung. Ziel ist die technisch konsistente Realisierung einer Visualisierungsanwendung, die präemptive Scheduling-Verfahren korrekt berechnet und verständlich darstellt.

5.1. Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als clientseitige Webanwendung mit Vue 3 und TypeScript umgesetzt. Vue.js Core Team [2026]. Vue strukturiert die Benutzeroberfläche komponentenbasiert, während TypeScript die fachlichen Datenmodelle typensicher abbildet. Der Build- und Entwicklungsprozess erfolgt mit Vite [2026], die automatisierte Verifikation mit Vitest Vitest Team [2026]. Für gezielte Übergänge in der Oberfläche wird GSAP eingesetzt GreenSock [2026].

Eine zentrale technische Entscheidung ist die Trennung von Simulationslogik, Zustandsverwaltung und Darstellung. Dadurch bleibt der Simulationskern unabhängig testbar, während die Oberfläche auf konsistenten Ergebnisdaten operiert.

5.1.1. Begründung der Technologieentscheidungen

Die Auswahl des Technologie-Stacks orientiert sich an den fachlichen und didaktischen Anforderungen der Anwendung. Da der Scheduling-Visualizer vollständig clientseitig arbeitet und unmittelbar auf Benutzerinteraktionen reagieren soll, wurde eine Webtechnologie gewählt, die komponentenbasierte Oberflächen, reaktive Zustandsaktualisierung und eine einfache Bereitstellung als statische Anwendung verbindet.

Vue 3 wurde als UI-Framework gewählt, weil die Oberfläche in klar abgegrenzte Komponenten für Szenariokonfiguration, Laufsteuerung, Visualisierung und Vergleichsansicht aufgeteilt werden kann. Vue.js Core Team [2026]. Die reaktive Aktualisierung sorgt dafür, dass Änderungen am aktiven Laufzustand direkt in den betroffenen Ansichten sichtbar werden. Die Composition API unterstützt dabei die Trennung wiederverwendbarer Zustands- und Interaktionslogik von der konkreten Darstellung. Dadurch lässt sich beispielsweise die Wiedergabesteuerung unabhängig von einer einzelnen Ansicht organisieren.

TypeScript ergänzt diese Struktur um statische Typisierung. Domänenobjekte wie Prozesse, Ereignisse, Segmente, Snapshots und Metriken bilden Schnittstellen zwischen Simu-

lationskern und Benutzeroberfläche. Eine explizite Typisierung macht diese Schnittstellen bereits während der Entwicklung überprüfbar und reduziert das Risiko, dass eine Ansicht unvollständige oder strukturell falsche Simulationsergebnisse verarbeitet. Dies unterstützt insbesondere das Prinzip der gemeinsamen Ergebnisbasis, das mit `SimulationRun` umgesetzt wird.

Vite wurde als Build- und Entwicklungswerkzeug eingesetzt Vite [2026]. Für die Entwicklung ist vor allem die kurze Rückkopplungszeit durch schnelle Aktualisierung relevant: Änderungen an Komponenten und Styles können unmittelbar geprüft werden. Für die Bereitstellung erzeugt Vite zudem statische Artefakte, die anschließend unabhängig von einer serverseitigen Anwendungslogik ausgeliefert werden können.

Für die Gantt-Darstellung und weitere zeitbezogene Ansichten wird SVG verwendet. SVG eignet sich für die präzise Positionierung von Zeitabschnitten, Segmentgrenzen, Beschriftungen und Ereignismarkierungen. Die einzelnen Elemente bleiben dabei im Dokumentmodell adressierbar und können mit CSS oder Ereignisbehandlung verbunden werden. Gegenüber einer reinen Canvas-Darstellung erleichtert dies die Zuordnung von Interaktionen und Beschriftungen zu konkreten Prozessen.

GSAP ergänzt die Darstellung für gezielte Übergänge und Animationen GreenSock [2026]. Die Bibliothek wird bewusst nur für Präsentationseffekte eingesetzt; sie verändert weder Scheduling-Entscheidungen noch die erzeugten Ergebnisdaten. Damit bleibt die fachliche Berechnung deterministisch, während Zustandswechsel in der Oberfläche zeitlich nachvollziehbar hervorgehoben werden können.

Vitest wurde für die automatisierte Verifikation der Simulationslogik gewählt Vitest Team [2026]. Die deterministische Ausführung der Verfahren ermöglicht reproduzierbare Tests von Ereignisfolgen, Präemptionen, Segmenten und Kennzahlen. Die Tests greifen dabei auf die fachlichen Ergebnisstrukturen zu und müssen die Darstellung nicht nachbilden. Dadurch werden fachliche Fehler unabhängig von Layout- oder Animationseffekten erkennbar.

Die Technologieentscheidungen bilden somit eine zusammenhängende Architektur: Vue strukturiert die Interaktion, TypeScript beschreibt die fachlichen Verträge, der Simulationskern erzeugt reproduzierbare Daten, SVG stellt diese präzise dar und Vitest überprüft die Kernlogik. Vite und GSAP unterstützen Entwicklungsprozess und Präsentation, ohne die fachliche Zuständigkeit des Simulationskerns zu erweitern.

5.2. Datenmodell und Simulationskern

Die fachliche Basis bildet ein einheitliches Prozessmodell mit Attributen wie Ankunftszeit, Burst-Zeit, Priorität, Restzeit und Status. Auf dieser Grundlage werden alle Verfahren innerhalb eines gemeinsamen Simulationsrahmens berechnet.

Ein Simulationslauf erzeugt mehrere zusammenhängende Ergebnisstrukturen:

- **Ereignislog:** chronologische Ereignisse (z. B. Dispatch, Präemption, Abschluss),
- **Segmente:** zeitlich abgegrenzte Ausführungsintervalle für die Gantt-Darstellung,
- **Snapshots:** Zustandsabbilder für Wiedergabe und Zeitsprung,
- **Metriken:** aggregierte Kennzahlen wie Warte-, Durchlauf- und Reaktionszeit sowie Kontextwechsel.

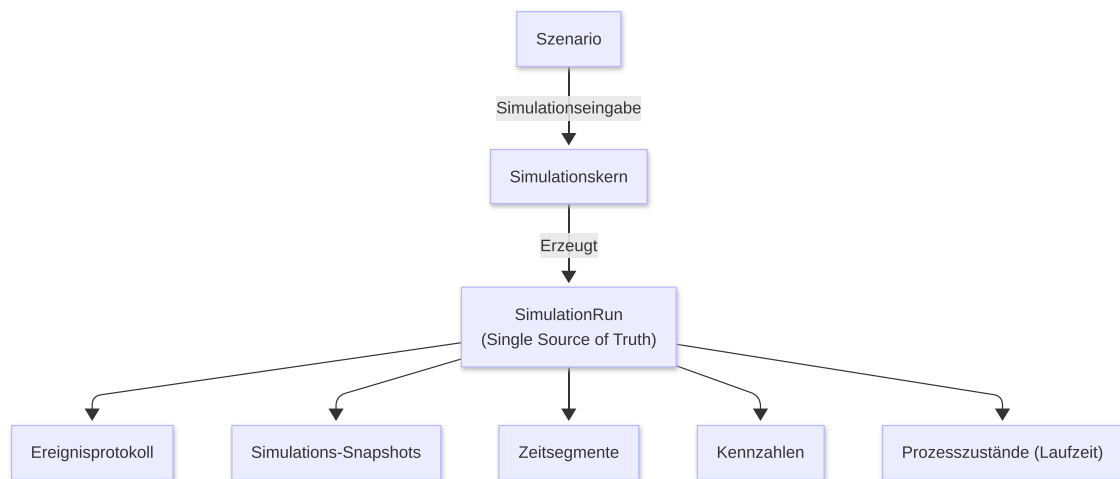


Abbildung 5.1.: Domänenmodell des Simulationslaufs mit seinen Ergebnisstrukturen.

Das Domänenmodell zeigt **SimulationRun** als zentrales Ergebnisobjekt. Es bündelt Ereignisse, Snapshots, Segmente, Kennzahlen und Prozesszustände und bildet damit die gemeinsame Datenbasis für nachgelagerte Ansichten.

5.2.1. Single Source of Truth: SimulationRun

Die Implementierung folgt dem Prinzip der *Single Source of Truth*: **SimulationRun** repräsentiert das vollständige Ergebnis eines Simulationsdurchlaufs. Nachgelagerte Darstellungen – insbesondere Gantt-Visualisierung, Vergleichsansichten und Kennzahlen – greifen ausschließlich auf dieses Objekt zu. Dadurch basieren Darstellung und Bewertung auf identischer Datenbasis.

SimulationRun umfasst den Ereignisverlauf (**events**), Zustandsabbilder (**snapshots**), Ausführungssegmente (**segments**), aggregierte Kennzahlen (**finalMetrics**) und Prozesszustände (**processStates**). Inkonsistenzen zwischen visualisiertem Ablauf und numerischer Auswertung werden dadurch vermieden.

Die zentrale Ergebnisstruktur unterstützt zugleich Reproduzierbarkeit und Testbarkeit: Bei identischem Szenario und identischen Parametern entsteht deterministisch dieselbe Struktur; Tests können direkt auf Ereignisse, Segmente und Kennzahlen zugreifen. Neue Ansichten lassen sich ergänzen, ohne den Simulationskern zu verändern.

5.3. Umsetzung der Scheduling-Verfahren

Round Robin, LCFS, Strict Priority und MLFQ wurden als präemptive Strategien innerhalb derselben Simulationslogik implementiert:

- **Round Robin:** Präemption nach Ablauf eines konfigurierbaren Zeitquantums.
- **LCFS (präemptiv):** Neu eintreffende Prozesse verdrängen den laufenden Prozess.
- **Strict Priority:** Auswahl des höchstpriorisierten bereiten Prozesses mit sofortiger Präemption bei höher priorisierten Neuankünften.
- **MLFQ:** Mehrere Warteschlangen mit stufenabhängigen Quantumswerten und Herabstufungslogik.

Alle Verfahren nutzen dieselbe Tick- und Ereignisverarbeitung. Unterschiede im Ablauf und in den Kennzahlen ergeben sich damit aus der Verfahrensstrategie, nicht aus unterschiedlicher Berechnungsinfrastruktur.

5.3.1. Ablauf eines Simulationslaufs

`simulateScenario()` erhält das konfigurierte Szenario, führt die tick-basierte Simulation aus und erzeugt ein vollständiges `SimulationRun`. Die Oberfläche rendert anschließend die enthaltenen Segmente, Snapshots, Ereignisse und Kennzahlen.

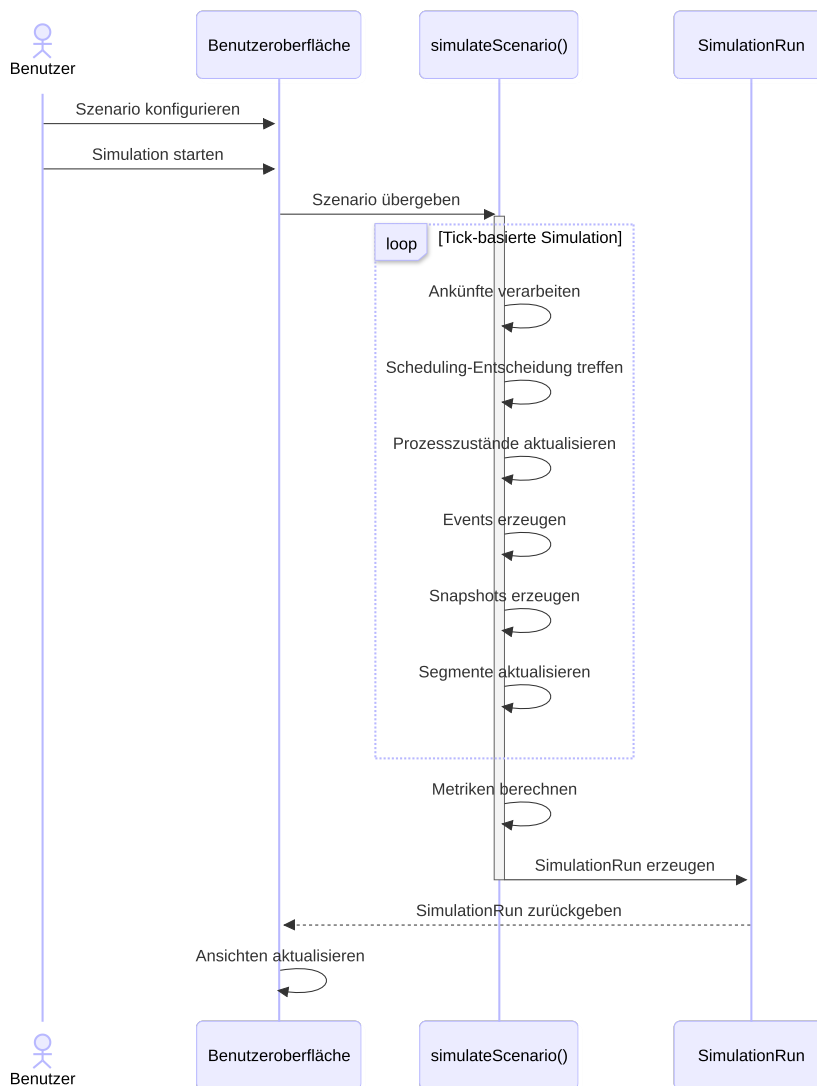


Abbildung 5.2.: Sequenzdiagramm eines Simulationslaufs von der Benutzeraktion bis zur Aktualisierung der Benutzeroberfläche.

Die tick-basierte Schleife verarbeitet Ankünfte, Scheduling-Entscheidungen, Präemptionen und Abschlüsse in zeitlicher Reihenfolge. Die erforderlichen Artefakte für Wiedergabe und Auswertung entstehen innerhalb dieses Ablaufs, sodass die Oberfläche auf einem vorberechneten Laufzustand navigieren kann.

5.3.2. Prozesszustände und Übergänge

Jeder Prozess durchläuft einen definierten Lebenszyklus. Die Zustände **pending**, **ready**, **running**, **preempted** und **finished** beschreiben, ob ein Prozess noch nicht eingetroffen, zur Ausführung bereit, aktiv, verdrängt oder abgeschlossen ist.

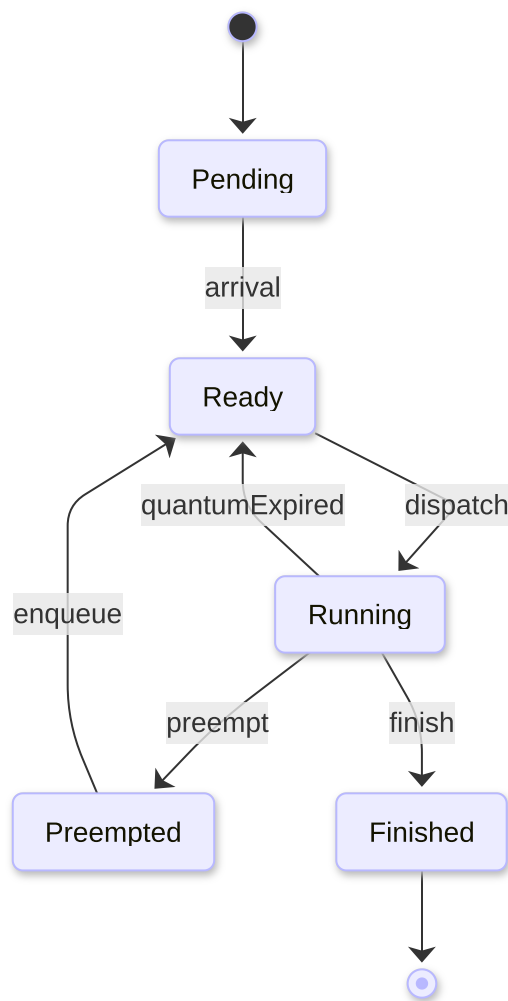


Abbildung 5.3.: Zustände und Übergänge eines Prozesses im Simulationsmodell.

Die Übergänge sind mit Ereignissen verknüpft: **arrival** (Eintritt), **dispatch** (CPU-Zuteilung), **preempt** (Verdrängung), **quantumExpired** (Ablauf des Zeitquantums) und **finish** (Abschluss). Die code-nahe Benennung erleichtert die Rückverfolgbarkeit zwischen Modell, Implementierung und Tests.

5.4. Umsetzung der Visualisierung

Die Visualisierung ist in getrennte Funktionsbereiche gegliedert: Eingabe und Konfiguration, Laufsteuerung, Fokusansicht, Kennzahlenbereich und Vergleichsansicht.

Die zeitliche Ausführung wird als Gantt-Darstellung auf Basis der Segmentdaten visualisiert. Ergänzende Ansichten zeigen Wartesituationen und laufzeitbezogene Kennzahlen. Die Umsetzung nutzt SVG-basierte Darstellungen, um Zeitachsen, Segmentgrenzen und Ereignisse präzise abzubilden.

Ziel ist fachliche Lesbarkeit: Entscheidungen des Schedulers sollen zeitlich und inhaltlich

nachvollziehbar werden.

5.5. Interaktionsmechanismen und Nutzerworkflow

Die Interaktion folgt einem didaktischen Ablauf: Szenario definieren, Verfahren auswählen, Lauf steuern, Ergebnis interpretieren und Verfahren vergleichen.

Die Wiedergabelogik arbeitet auf vorberechneten Snapshots und startet die Simulation nicht bei jedem Interaktionsschritt neu. Dadurch bleiben Funktionen wie Start, Pause, Schrittmodus, Geschwindigkeitsanpassung und Zeitsprung reaktiv, ohne die Konsistenz der Ergebnisdaten zu gefährden.

Im Vergleichsmodus werden mehrere Läufe auf identischer Eingabebasis nebeneinander dargestellt. Das unterstützt die direkte Gegenüberstellung algorithmischer Unterschiede.

5.6. Umsetzung der Accessibility-Anforderungen

Die Dialogkomponenten verwenden mit `useFocusTrap` eine gemeinsame Implementierung für Escape, Tab-Zirkulation, initialen Fokus, `aria-hidden/inert` auf dem Hauptinhalt und Fokus-Rückgabe. Der Ranking-Dialog wurde an dieses Muster angeschlossen. Dadurch wird der Dialog nicht nur visuell überlagert, sondern auch als modal begrenzter Interaktionszustand behandelt.

Die Vergleichsansicht in `ComparisonPanel.vue` ist als Tablist mit `role=tab`, `aria-selected`, `aria-controls` und Tabpanels umgesetzt. Pfeil-, Home- und End-Tasten verschieben den Fokus innerhalb der Auswahl. In `GanttWithGsap.vue` und `Scrubber.vue` sind die Zeitregler sichtbar beschriftet und mit dem live aktualisierten Zeitstatus verbunden.

`GanttWithGsap.vue` ist die im `AppShell` verwendete produktive Gantt-Komponente. Sie bündelt Playback, Slider, Statuskommunikation, GSAP-Übergänge und die Reduced-Motion-Präferenz. Für die eigentliche SVG-Struktur rendert sie intern die spezialisierte Komponente `Gantt.vue`. Dort werden Titel, Beschreibung, Zusammenfassung und zugängliche Namen der einzelnen Prozesssegmente erzeugt. Bei geteilten Segmenten bleibt nur ein Teil ein eigenständiges Tastaturziel, damit dieselbe fachliche Information nicht doppelt angeboten wird. Das Burger-Menü unterstützt Escape, Erstfokus und Fokus-Rückgabe. Nicht notwendige GSAP- und Hover-Effekte werden bei aktivierter Präferenz für reduzierte Bewegung übersprungen; Änderungen dieser Präferenz werden während der Laufzeit berücksichtigt.

5.7. Build- und Deploymentprozess

Für die Bereitstellung wird ein containerbasierter Deploymentprozess verwendet. Die Anwendung wird mit Vite in statische Artefakte überführt und in ein Docker-Image paketi­ert. Dadurch entsteht eine reproduzierbare Laufzeitumgebung unabhängig vom lokalen Entwicklungsrechner.

Das Deployment erfolgt auf einer Hosting-Plattform mit Container-Unterstützung. Änderungen können konsistent gebaut und in stabiler Umgebung veröffentlicht werden; im Lehrkontext stellt dies eine einheitliche Versionsbasis für Demonstration und Nutzung sicher.

5.8. Besondere technische Herausforderungen

Eine zentrale Herausforderung war die Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Präemptive Verfahren erzeugen häufige Zustandswechsel, während die Oberfläche gleichzeitig unmittelbar auf Interaktionen reagieren muss.

Ein weiterer Schwerpunkt war die Sicherung der Deterministik durch klare Zustandsübergänge, konsistente Datenmodelle und testbare Schnittstellen. Zusätzlich ergibt sich ein Zielkonflikt zwischen Detailtiefe und Ressourcenbedarf: Dichtere Snapshot-Folgen verbessern Analyse und Zeitsprung, erhöhen jedoch Speicher- und Verarbeitungsaufwand. Dieser Konflikt wurde durch parametrisierbare Granularität der Wiedergabedaten adressiert.

6. Qualitätssicherung

Für eine didaktisch ausgerichtete Anwendung mit fachlichem Anspruch ist Qualitätssicherung ein zentraler Entwicklungsbestandteil. Bewertet werden nicht nur technische Funktionsfähigkeit, sondern insbesondere Konsistenz der Simulationslogik, Reproduzierbarkeit der Ergebnisse und Verlässlichkeit der Interaktion.

6.1. Qualitätsziele und Prüfstrategie

Die Qualitätssicherung verfolgt drei Hauptziele:

- **Fachliche Korrektheit:** Scheduling-Entscheidungen und Kennzahlen müssen konsistent abgeleitet werden.
- **Reproduzierbarkeit:** Identische Eingaben müssen identische Abläufe erzeugen.
- **Bedienbarkeit und Zugänglichkeit:** Zentrale Funktionen müssen verlässlich und verständlich nutzbar sein.

Zur Erreichung dieser Ziele wurde ein mehrstufiges Vorgehen gewählt: automatisierte Tests der Kernlogik, Prüfung zentraler Interaktionspfade sowie Bewertung grundlegender Accessibility-Aspekte.

6.2. Automatisierte Verifikation der Simulationslogik

Die Simulationslogik wird mit Vitest automatisiert geprüft Vitest Team [2026]. Im Fokus stehen deterministische Berechnungsschritte, korrekte Präemptionsentscheidungen und konsistente Kennzahlenableitung.

Geprüft werden unter anderem:

- Quantum-Verhalten und Wiedereinreihung bei Round Robin,
- Präemption bei Neuankunft in präemptiven Verfahren (z. B. LCFS, Strict Priority),
- Herabstufungslogik in MLFQ,
- Konsistenz von Ereignisfolge, Segmenten und Endzustand.

Zusätzlich wird validiert, ob unvollständige bzw. ungültige Eingaben robust normalisiert werden und erzeugte Segmente gültige Zeitgrenzen aufweisen.

6.3. Prüfung von Interaktion und Accessibility

Ergänzend zur Fachlogik wurde die Oberfläche hinsichtlich Bedienbarkeit und Zugänglichkeit geprüft. Betrachtet wurden die in Tabelle 3.1 definierten A11Y-Anforderungen. Als Referenz dient WCAG 2.1 W3C [2018]; die Prüfung ist auf den Scheduler fokussiert und keine vollständige Konformitätsbewertung.

Für den Lehrkontext ist wesentlich, dass Benutzerinnen und Benutzer ohne zusätzliche Hürden zwischen Szenariokonfiguration, Ablaufsteuerung und Ergebnisinterpretation wechseln können. Klare Statusanzeigen und eindeutige Steuerzustände unterstützen diese Nutzbarkeit.

6.4. Reproduzierbarkeit und Vergleichbarkeit

Ein zentrales Qualitätsmerkmal ist die deterministische Reproduzierbarkeit: Bei identischem Szenario und identischen Parametern entsteht derselbe Ablauf mit denselben Kennzahlen. Diese Eigenschaft ist für belastbare Vergleichsvergleiche wesentlich, da Unterschiede eindeutig auf die jeweilige Scheduling-Strategie zurückgeführt werden können.

Auch die Vergleichsansicht profitiert davon: Mehrere Läufe lassen sich auf gemeinsamer Eingabebasis direkt gegenüberstellen, ohne dass zufällige Abweichungen die Interpretation verzerren.

6.4.1. Automatisierte und statische Prüfungen

Die automatisierten Tests werden mit Vitest ausgeführt. Im aktuellen Stand bestehen zehn Testdateien mit 35 Tests. Davon prüfen die beiden Accessibility-Dateien insgesamt elf statische Marker- und Strukturbedingungen. Sie lesen Quelltext und prüfen beispielsweise Dialogrollen, Fokusmanagement, Tabsemantik, Sliderbeschriftungen, Gantt-Beschreibungen und Reduced-Motion-Gating. Der Test mit dem Dateinamen `axe.runtime.test.ts` führt keinen DOM-Scan mit `axe-core` aus und wird deshalb nicht als axe-Runtime-Nachweis gezählt. Der vorhandene Placeholder-Test bleibt als Scaffold erhalten und wird ebenfalls nicht als Qualitätsnachweis gezählt.

6.4.2. Manuelles Prüfprotokoll

Die manuelle Prüfung umfasst einen Keyboard-only-Durchlauf, Dialogöffnung und Escape, Fokusfalle und Fokus-Rückgabe, Burger-Menü, Tab-Navigation, Slider, Playback, Gantt-Segmente und Statusmeldungen. Zusätzlich werden Desktop- und schmale Viewports, 200

6.5. Erreichte Testabdeckung und Grenzen

Zum Zeitpunkt der Prüfung umfasst die automatisierte Testsuite zehn Testdateien mit insgesamt 35 erfolgreichen Tests. Damit werden zentrale Kern- und Integrationspfade sowie statische Accessibility-Merkmale abgedeckt und die technische Verlässlichkeit der Implementierung erhöht.

Die Testsuite stellt jedoch keinen vollständigen formalen Korrektheitsbeweis für alle denkbaren Prozesskonstellationen und keinen vollständigen WCAG-/BITV-Audit dar. Ebenso ersetzt sie keine empirische Untersuchung der Lernwirksamkeit mit Nutzergruppen. Die Qualitätssicherung ist daher als begrenzter, risikoorientierter Prüfstand zentraler Funktionen zu verstehen.

6.6. Fazit zur Qualitätssicherung

Qualitätssicherung wurde als integraler Bestandteil der Entwicklung umgesetzt. Die Kombination aus deterministischer Simulationslogik, automatisierter Verifikation und strukturierter Interaktionsprüfung bildet die Grundlage für ein fachlich belastbares und didaktisch nutzbare Lehrwerkzeug.

7. Ergebnis und Produktpräsentation

Dieses Kapitel stellt das entwickelte Produkt vor und ordnet es anhand der zuvor definierten Ziele ein. Im Fokus stehen Funktionsumfang, Nutzungsablauf und fachliche Belastbarkeit der Anwendung als didaktisches Werkzeug.

7.1. Entwickeltes Endprodukt

Das Ergebnis der Arbeit ist eine interaktive Webanwendung zur Visualisierung präemptiver CPU-Scheduling-Verfahren. Sie ermöglicht die Definition von Prozessen mit relevanten Parametern, die Ausführung unterschiedlicher Verfahren und die vergleichende Analyse von Ablauf und Kennzahlen.

Die entwickelte Anwendung ist als öffentlich zugängliches Endprodukt verfügbar:

- Live-Anwendung: <https://scheduling-visualizer.onrender.com/>
- Quellcode und technische Dokumentation: <https://github.com/cherrydude/scheduling-visualizer>

Die Links wurden am 18.09.2026 geprüft. Da sich die Live-Anwendung und der Quellcode nach Abschluss der Arbeit weiterentwickeln können, dokumentieren die Screenshots den im Rahmen dieser Arbeit untersuchten Entwicklungsstand.

Das Produkt vereint drei Kerneigenschaften:

- konsistente Simulationslogik,
- interaktive und verständliche Ablaufdarstellung,
- direkte Vergleichbarkeit mehrerer Verfahren auf identischer Datenbasis.

Damit geht die Anwendung über eine reine Demonstration hinaus und ist als Werkzeug für Lehr- und Lernszenarien einsetzbar.

7.2. Produktüberblick und Nutzerfluss

Der typische Nutzungsablauf folgt einer festen Sequenz:

1. Definition oder Auswahl eines Szenarios,
2. Auswahl und Parametrisierung eines Scheduling-Verfahrens,

3. Ausführung und Steuerung des Laufs,
4. Interpretation von Ablauf und Kennzahlen,
5. Vergleich mit alternativen Verfahren.

Dieser Ablauf unterstützt das didaktische Ziel, Entscheidungen nicht nur als Endwerte, sondern im zeitlichen Kontext ihrer Entstehung zu analysieren.

7.3. Produktpräsentation mit Screenshots

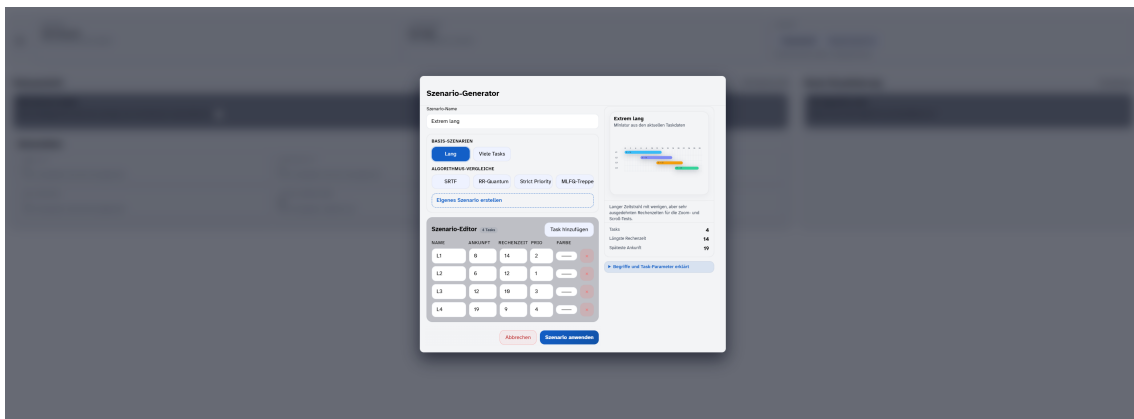


Abbildung 7.1.: Geladenes Szenario mit sichtbaren Task-Parametern als Grundlage für die Simulation.

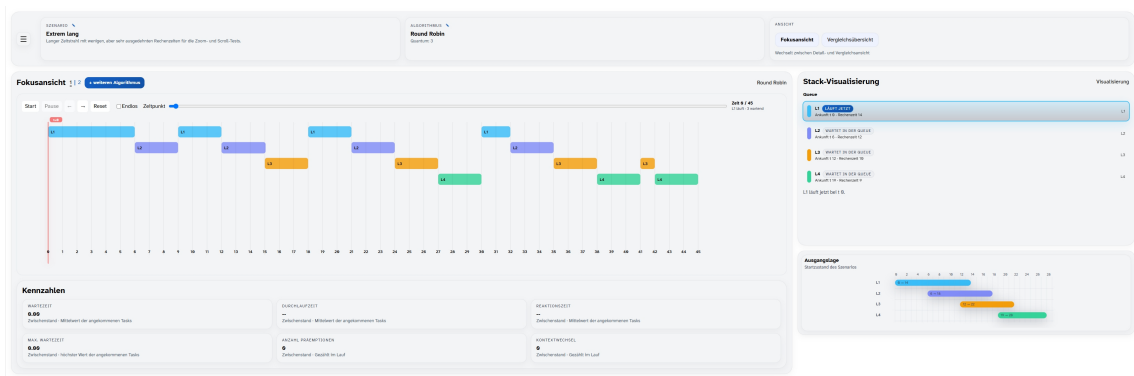


Abbildung 7.2.: Fokusansicht mit zeitlicher Ablaufdarstellung und Kennzahlenbereich für einen einzelnen Lauf.

7.4. Umgesetzte Funktionalität im Soll-Ist-Abgleich

Die aus Kapitel 3 abgeleiteten Anforderungen wurden in den zentralen Funktionsbereichen umgesetzt:

- **Szenarioverwaltung:** Erfassung und Bearbeitung von Task-Daten mit relevanten Scheduling-Parametern.



Abbildung 7.3.: Vergleichsansicht mehrerer Verfahren auf identischer Szenariobasis.

- **Verfahrensauswahl:** Ausführung von Round Robin, LCFS, Strict Priority und MLFQ innerhalb eines einheitlichen Simulationsrahmens.
- **Ablaufsteuerung:** Start, Pause, Schrittmodus, Zeitsprung und Geschwindigkeitsanpassung.
- **Visualisierung:** Zeitliche Darstellung der Ausführung mit Ereignisbezug.
- **Kennzahlenanalyse:** Ausgabe zentraler Metriken pro Lauf.
- **Vergleichsmodus:** Gegenüberstellung mehrerer Läufe unter identischen Eingaben.

Damit ist der funktionale Kern vollständig umgesetzt. Nicht-funktionale Ziele wie Deterministik, Nachvollziehbarkeit und Erweiterbarkeit werden durch Architektur und Implementierung unterstützt.

7.5. Accessibility-relevante Produktzustände

Die Produktnachweise sind nicht nur als visuelle Screenshots zu verstehen. Die Fokusansicht unterstützt die Handlung, den Ablauf über Start, Pause, Schrittsteuerung und Zeitregler zu untersuchen; der Regler besitzt eine sichtbare Beschriftung und einen angekündigten Zeitstatus. Der Gantt-Nachweis zeigt Prozesssegmente, Zeitgrenzen und Ereignismarkierungen. Ein Segment kann per Tastatur fokussiert und aktiviert werden, sodass die Ablaufanalyse nicht ausschließlich von Pointer-Interaktion abhängt.

Der Dialognachweis steht für einen begrenzten Interaktionszustand: Beim Öffnen wird der Fokus in den Dialog geführt, Tab bleibt innerhalb des Dialogs und Escape schließt ihn. Nach dem Schließen kehrt der Fokus zum auslösenden Element zurück. Die Vergleichsansicht stellt Anwendungsfälle als Tabs dar; der aktive Zustand wird nicht nur farblich, sondern auch über Auswahlstatus und zugehöriges Panel ausgedrückt. Statusmeldungen zu bestem und schwächstem Lauf werden zusätzlich über einen Statuskanal bereitgestellt.

Diese Zustände belegen konkrete Nutzerhandlungen und Implementierungsentscheidungen. Sie sind zugleich nur ein fokussierter Nachweis für die in Tabelle 3.1 definierten Anforderungen und keine Aussage über vollständige WCAG- oder BITV-Erfüllung.

7.6. Ergebnisbetrachtung

Das entwickelte Produkt erfüllt die Zielsetzung der Arbeit: Präemptive Scheduling-Verfahren werden nicht nur berechnet, sondern als dynamischer Ablauf verständlich und vergleichbar dargestellt. Die Verknüpfung von Ablaufvisualisierung, Kennzahlen und Interaktion ermöglicht die Analyse algorithmischer Unterschiede in einem didaktisch nutzbaren Kontext.

Besonders relevant ist die Sichtbarkeit der Entscheidungspfade. Benutzerinnen und Benutzer erkennen nicht nur das Ergebnis, sondern auch dessen Entstehung im zeitlichen Verlauf. Daraus ergibt sich ein praktischer Mehrwert für Lehre, Demonstration und eigenständiges Lernen.

8. Fazit und Ausblick

Die vorliegende Arbeit zeigt, dass sich präemptive CPU-Scheduling-Verfahren in einer interaktiven Webanwendung fachlich konsistent und didaktisch wirksam aufbereiten lassen. Die zentrale Leistung liegt in der Verbindung von Simulationslogik, Visualisierung und steuerbarer Interaktion zu einem integrierten Lehrwerkzeug.

Ausgehend von den theoretischen Grundlagen wurden Anforderungen an Fachlogik, Usability und Transparenz abgeleitet und in ein umsetzbares Architekturmodell überführt. Die Implementierung realisiert diese Anforderungen in einer modularen Struktur mit klarer Trennung von Simulationskern und Darstellung. Dadurch bleiben Berechnungen nachvollziehbar, testbar und unter identischen Bedingungen reproduzierbar.

Die Anwendung erfüllt damit die Zielsetzung der Arbeit: Scheduling-Entscheidungen werden sichtbar, zeitliche Abläufe mit Kennzahlen verknüpft und Verfahren auf gemeinsamer Szenariobasis direkt vergleichbar gemacht. Der didaktische Mehrwert liegt insbesondere in der Darstellung von Entscheidungspfaden statt reiner Endergebnisse.

Die Qualitätssicherung stützt diese Einordnung durch automatisierte Tests der Kernlogik, die Prüfung zentraler Interaktionspfade sowie die Berücksichtigung grundlegender Accessibility-Anforderungen. Damit ist die Anwendung sowohl als technische Umsetzung als auch als Lerninstrument belastbar einsetzbar.

8.1. Ausblick

Für die Weiterentwicklung ergeben sich mehrere Richtungen. Erstens kann die fachliche Breite durch zusätzliche Scheduling-Verfahren und differenziertere Lastmodelle erweitert werden. Zweitens lässt sich die Analyse durch weitere Kennzahlen, Exportfunktionen und ergänzende Vergleichsmodi vertiefen. Drittens sind didaktische Erweiterungen wie geführte Lernpfade, Aufgabenserien und vorbereitete Szenariosets für den Einsatz in Lehrveranstaltungen sinnvoll.

Darüber hinaus ist eine empirische Evaluation mit Studierenden ein naheliegender nächster Schritt, um den Lerneffekt systematisch zu untersuchen. Die vorliegende Arbeit schafft dafür eine tragfähige technische und konzeptionelle Grundlage.

Literaturverzeichnis

Computer History Museum. Compatible time-sharing system (ctss) is demonstrated, 1961.
URL <https://www.computerhistory.org/timeline/1961/>.

GreenSock. Gsap documentation, 2026. URL <https://greensock.com/docs/>.

Andreas M. Heinecke and Jens Gerken. *Mensch-Computer-Interaktion: Basiswissen für Entwickler und Gestalter*. Xpert.press. Springer Vieweg, Berlin/Heidelberg, 3 edition, 2024. ISBN 978-3-662-67568-7.

Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3): 259–290, 2002. doi: 10.1006/jvlc.2002.0237.

William Stallings. *Betriebssysteme: Prinzipien und Umsetzung*. Pearson Studium, München, 4., überarb. auflage edition, 2003.

Andrew S. Tanenbaum and Herbert Bos. *Moderne Betriebssysteme*. Pearson Studium, München, 4., aktualisierte auflage edition, 2016.

Vite. Vite documentation, 2026. URL <https://vitejs.dev/>.

Vitest Team. Vitest documentation, 2026. URL <https://vitest.dev/>.

Vue.js Core Team. Vue 3 documentation, 2026. URL <https://vuejs.org/>.

W3C. Web content accessibility guidelines (wcag) 2.1, 2018. URL <https://www.w3.org/TR/WCAG21/>.

A. Anhang

A.1. Ergänzende Abbildungen

A.2. Zusätzliche Testfälle und Szenarien

A.3. Operationalisierungsmatrix Accessibility

Die vollständige Matrix ergänzt die Kurzfassung aus Tabelle 3.1 um Implementierungsort, Nachweis und Ergebnisstatus.

ID	Entwurfsentscheidung	Implementierung/Nachweis	Ergebnis und Grenze
A11Y-01	Semantische Dialoge, Namen und Panels.	Dialogkomponenten, Tablist, Statusrollen; statischer Test.	Erfüllt im fokussierten Umfang; Screenreader-Matrix offen.
A11Y-02	Keyboard-first-Bedienung.	Menü, Tabs, Gantt und Playback; manueller Walkthrough.	Geprüft; Browser-/Assistive-Tech-Kombinationen begrenzt.
A11Y-03	Sichtbarer Fokus und Dialogfalle.	<code>useFocusTrap</code> , Ranking-Dialog; Code- und manueller Test.	Erfüllt für die geprüften Dialogpfade.
A11Y-04	Statuskanäle statt ausschließlich visueller Effekte.	Live-Status in Playback und Vergleich; manueller Test.	Erfüllt im geprüften Ablauf.
A11Y-05	Beschreibbares, interaktives SVG.	<code>Gantt.vue</code> , Segmentlabels; statisch und manuell.	Erfüllt im fokussierten SVG; keine vollständige AT-Abdeckung.
A11Y-06	Nichtfarbliche Zustandsmerkmale und Kontrastprüfung.	Statuslabels, Fokusregeln, <code>Forced Colors</code> ; manueller Checker.	Zustände geprüft; freie Prozessfarben bleiben Grenze.
A11Y-07	Reflow und Zoom.	Schmalen Viewport und 200 % Zoom; manuelles Protokoll.	Ergebnis nach Durchführung eintragen; keine Geräteabdeckung.
A11Y-08	Reduzierte Bewegung.	Medienpräferenz in <code>GanttWithGsap.vue</code> ; manueller Test.	Nicht notwendige Effekte reduziert; keine Nutzerstudie.

A.4. Kompaktes manuelles Accessibility-Prüfprotokoll

Setup	Prüfschritte	Beobachtung/Status
Desktop, Tastatur	Dialog öffnen, Tab/Shift-Tab, Escape, erneut fokussieren.	Fokusfalle und Rückgabe dokumentieren: behoben/offen.
Desktop, Tastatur	Menü, Tabs, Slider, Playback und Gantt ohne Maus bedienen.	Reihenfolge, Aktivierung und Status dokumentieren.
Schmalere Viewport, 200 %	Vergleich, Gantt und Steuerung bei vergrößerter Darstellung prüfen.	Überlappung, Abschneiden und Reflow dokumentieren.
Reduced Motion/Forced Colors	Präferenz aktivieren und Zustände erneut ausführen.	Nicht notwendige Bewegung und Fokuskontrast dokumentieren.
Kontrastprüfung	Tatsächlich verwendete Text-, Fokus- und Statuszustände messen.	Messwert und verbleibende Farbrisiken dokumentieren.

A.5. Ergänzende technische Artefakte (z. B. Konfigurationsbeispiele)

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese BACHELORARBEIT selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Marten Kirschenhofer